НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»

Навчально-науковий інститут комп’ютерних систем

Кафедра інженерії програмного забезпечення

КУРСОВА РОБОТА

з ОБ’ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ

на тему: Ріелторська фірма

Студента 2 курсу АС-242 групи

спеціальності F2 (121) Інженерія

програмного забезпечення

Заїка Д.В.

(прізвище та ініціали)

Керівник _____ ств. Шаманіна Т.В.

Національна шкала _____

Кількість балів: ____Оцінка: ECTS____

Одеса - 2025

Зміст

1. Варіант завдання……..……..3
2. Вступ…….……..4
3. Постанова задачі……5
4. Аналіз предметної області……...7
   4.1. Опис діяльності ріелторської фірми……...…7
   4.2. Основні сутності та їх взаємозв'язки (Ріелтор, Клієнт, Нерухомість).....7
   4.3. Опис класів, методів, інтерфейсів (Realtor, Client, Property, Agency, ServiceType, AvailabilityStatus).....9
   4.4. Загальна структура програми…….…11
5. UML-діаграми……..13
   5.1. Діаграма варіантів використання (Use Case Diagram).....13
   5.2. Діаграма класів (Class Diagram).....15
   5.3. Діаграма послідовностей (Sequence Diagram).....17
6. Реалізація програми на Java……...19
   6.1. Опис середовища розробки……..…..19
   6.2. Структура та взаємодія класів……....20
   6.3. Графічний інтерфейс та основні операції (три вкладки).....27
   6.4. Опис додаткових можливостей програми…….……29
7. Тестування програми……..31
   7.1. CRUD-операції для ріелтора…….…..31
   7.2. CRUD-операції для клієнта……..……..35
   7.3. CRUD-операції для нерухомості……...……42
   7.4. Запити до системи (списки, середня ціна, попит тощо) ……49
8. Висновки……..56
   8.1. Досягнуті результати……...56
   8.2. Труднощі та їх вирішення…….……..58
9. Список використаних джерел……..…..60
10. Додаток А……62

1.Варіант завдання

Варіант 2. Ріелторська фірма

Властивості	Класи		
	Ріелтор	Клієнт	Нерухомість
Назва фірми	+	+	
ПІБ	+	+	
Адреса	+		+
№ телефону	+	+	
Тип послуги		+	
Площа			+
Поверх			+
Рік побудови			+
Кількість кімнат			+
Вартість			+
Id нерухомості		+	+

Запити до системи:

1. Список ріелторів.
2. Список адрес продажу.
3. Список клієнтів.
4. Площа, що має найбільший попит.
5. Нерухомість з найбільшою кількістю кімнат.
6. Середня вартість нерухомості.

2. Вступ

У сучасному світі інформаційні технології відіграють ключову роль у автоматизації бізнес-процесів, зокрема в сфері нерухомості. Ріелторські фірми щоденно працюють з великим обсягом даних про об'єкти нерухомості, клієнтів та угоди, що потребує ефективного програмного забезпечення для обліку, пошуку та аналізу інформації.

Курсова робота присвячена розробці програмного продукту для ріелторської фірми з використанням принципів об'єктно-орієнтованого програмування (ООП) на мові Java. Основна мета полягає у створенні структури класів, яка відображає реальні сутності предметної області, забезпечує логіку взаємодії між ними та реалізує базову функціональність системи.

Особистий інтерес до теми зумовлений бажанням глибше опанувати Java як мову професійної розробки, а також застосувати теоретичні знання з ООП на практиці. Завдання курсової дозволяє не лише реалізувати технічну частину, а й структурувати логіку програми, що є важливим кроком у формуванні інженерного мислення.

У процесі розробки застосовано ключові концепції ООП — інкапсуляцію, спадкування, поліморфізм, а також побудовано UML-діаграми, що ілюструють варіанти використання та структуру класів. Робота включає аналіз предметної області, постановку задачі, реалізацію програми, тестування та висновки щодо отриманих результатів. Практичне значення полягає у створенні системи, яка може бути використана як основа для подальшого розвитку програмних засобів у сфері нерухомості.

3. Постанова задачі

Метою курсової роботи є розробка програмного забезпечення для ріелторської фірми, яке дозволяє автоматизувати облік об'єктів нерухомості, клієнтів та ріелторів. Програма має забезпечити зручну взаємодію між основними сутностями предметної області, а також реалізувати базові операції, необхідні для роботи фірми.

Основні завдання:

- Створити об'єктно-орієнтовану модель предметної області, яка включає класи для представлення рієлторів, клієнтів та нерухомості.
- Реалізувати функціональність додавання, редагування, перегляду та видалення записів для кожної сутності.
- Забезпечити можливість збереження даних у файл у форматі CSV та їх подальшого відновлення.
- Реалізувати набір запитів до системи: список ріелторів, список клієнтів, список адрес продажу, нерухомість з найбільшою кількістю кімнат, середня вартість нерухомості, визначення об'єктів з найбільшим попитом.
- Побудувати UML-діаграми, що відображають варіанти використання та структуру класів.
- Написати програму мовою Java з використанням принципів ООП: інкапсуляції, успадкування, поліморфізму.
- Провести тестування роботи програми на прикладах.

Вимоги до системи:

- Програма повинна мати графічний інтерфейс на базі Java Swing з трьома вкладками (Рієлтор, Клієнт, Нерухомість).
- Дані зберігаються у вигляді списків (ArrayList) та можуть експортуватися у CSV-файли.
- Кожен клас має бути чітко обгрунтований, з відповідними методами та атрибутами.
- Код має бути прокоментований, а структура — логічно узгоджена з UML-діаграмами.

Очікуваний результат:

Створення працюючої програми, яка демонструє правильне застосування принципів ООП, забезпечує автоматизацію обліку даних у ріелторській фірмі, формує статистичні звіти та може бути використана як основа для подальшого розвитку системи.

4. Аналіз предметної області

4.1. Опис діяльності ріелторської фірми.

Ріелторська фірма — це організація, яка займається посередництвом у купівлі, продажу та оренді об'єктів нерухомості. Основними учасниками процесу є клієнти (покупці та орендарі), ріелтори (посередники, що представляють фірму), а також об'єкти нерухомості, які мають певні характеристики: тип, адреса, площа, кількість кімнат, рік побудови, ціна та статус (на продаж, продана, в оренді). Програмна система моделює ці сутності та забезпечує базову функціональність:

- додавання нових об'єктів нерухомості, клієнтів та ріелторів;
- редагування та видалення існуючих записів;
- пошук і перегляд даних за параметрами;
- збереження інформації у файл у форматі CSV;
- виконання запитів і формування статистики (середня вартість, найбільша кількість кімнат, найпопулярніша площа тощо).

Таким чином, діяльність ріелторської фірми відображається у програмі через три основні сутності, що взаємодіють між собою, а система автоматизує облік і аналіз даних, підвищуючи ефективність роботи агентства.

4.2. Основні сутності та їх взаємозв'язки (Ріелтор, Клієнт, Нерухомість).

У програмі для ріелторської фірми виділено три основні сутності: Ріелтор (Realtor), Клієнт (Client) та Нерухомість (Property). Їх взаємодія організована через клас Agency, який виступає менеджером даних, забезпечує виконання CRUD-операцій та формування статистики.

- **Об'єкт нерухомості (Property)**

  Атрибути:

  - type — тип нерухомості (квартира, будинок, офіс)
  - location — район або локація
  - address — адреса об’єкта
  - area — площа у квадратних метрах
  - floor — поверх
  - yearBuilt — рік побудови

- rooms — кількість кімнат
- price — вартість об’єкта
- availability — статус (на продаж, продана, оренда)
- interestCount — кількість зацікавлень (попит)

Методи:

- markInterest() — збільшує показник попиту
- геттери/сеттери для кожного поля
- toString() — текстове представлення об’єкта

- **Клієнт (Client)**

  Атрибути:

  - id — унікальний ідентифікатор клієнта
  - fullName — ПІБ клієнта
  - address — адреса
  - phone — номер телефону
  - serviceType — тип послуги (купівля, оренда, підбір)

  Методи:

  - геттери/сеттери для кожного поля
  - toString() — текстове представлення клієнта

- **Ріелтор (Realtor)**

  Атрибути:

  - firmName — назва фірми
  - fullName — ПІБ ріелтора
  - address — адреса
  - phone — номер телефону
  - serviceType — тип послуги

  Методи:

  - геттери/сеттери для кожного поля
  - toString() — текстове представлення ріелтора

- **Ріелторська фірма (Agency)**

  Атрибути:

- properties — список об’єктів нерухомості
- clients — список клієнтів
- realtors — список ріелторів

Методи:

- addProperty(), removeProperty() — додавання/видалення нерухомості
- addClient(), removeClient() — додавання/видалення клієнтів
- addRealtor(), removeRealtor() — додавання/видалення ріелторів
- savePropertiesToFile() — збереження даних у файл CSV
- averagePrice() — обчислення середньої вартості нерухомості
- propertyWithMostRooms() — пошук об’єкта з найбільшою кількістю кімнат
- mostPopularArea() — визначення найпопулярнішої локації
- mostDemandedProperties() — сортування об’єктів за попитом

4.3. Опис класів, методів, інтерфейсів (Realtor, Client, Property, Agency, ServiceType, AvailabilityStatus).

У програмі реалізовано набір класів та перерахувань, які моделюють предметну область ріелторської фірми.

- **Клас Realtor**

  Атрибути:

  - firmName — назва фірми
  - fullName — ПІБ ріелтора
  - address — адреса
  - phone — номер телефону
  - serviceType — тип послуги (значення з перерахування ServiceType)

  Методи:

  - геттери/сеттери для кожного поля
  - toString() — текстове представлення ріелтора

- **Клас Client**

  Атрибути:

  - id — унікальний ідентифікатор клієнта

- fullName — ПІБ клієнта
- address — адреса
- phone — номер телефону
- serviceType — тип послуги (значення з перерахування ServiceType)

Методи:

- геттери/сеттери для кожного поля
- toString() — текстове представлення клієнта

- **Клас Property**

  Атрибути:

  - type — тип нерухомості (квартира, будинок, офіс)
  - location — район або локація
  - address — адреса об’єкта
  - price — вартість
  - availability — статус (значення з перерахування AvailabilityStatus)
  - area — площа
  - floor — поверх
  - yearBuilt — рік побудови
  - rooms — кількість кімнат
  - interestCount — кількість зацікавлень (попит)

  Методи:

  - markInterest() — збільшує показник попиту
  - геттери/сеттери для кожного поля
  - toString() — текстове представлення об’єкта

- **Клас Agency**

  Атрибути:

  - properties — список об’єктів нерухомості
  - clients — список клієнтів
  - realtors — список ріелторів

  Методи:

  - addProperty(), removeProperty() — додавання/видалення нерухомості

- addClient(), removeClient() — додавання/видалення клієнтів
- addRealtor(), removeRealtor() — додавання/видалення ріелторів
- savePropertiesToFile() — збереження даних у файл CSV
- averagePrice() — обчислення середньої вартості нерухомості
- propertyWithMostRooms() — пошук об’єкта з найбільшою кількістю кімнат
- mostPopularArea() — визначення найпопулярнішої локації
- mostDemandedProperties() — сортування об’єктів за попитом

- **Перерахування ServiceType**

  Визначає тип послуги, яку надає ріелтор або замовляє клієнт.

  Можливі значення:

  - BUY — купівля
  - RENT — оренда
  - CONSULT — консультація / підбір

- **Перерахування AvailabilityStatus**

  Визначає статус нерухомості.

  Можливі значення:

  - FOR_SALE — на продаж
  - SOLD — продана
  - RENTED — здана в оренду

4.4. Загальна структура програми

Програма побудована за принципами об’єктно-орієнтованого програмування. Кожна сутність предметної області реалізована у вигляді окремого класу (**Realtor, Client, Property**), що відповідає за власну логіку та атрибути. Взаємодія між класами здійснюється через методи, а дані зберігаються у колекціях типу **ArrayList**, що забезпечує динамічне керування списками об’єктів.

Центральним елементом є клас **Agency**, який виступає менеджером даних. Він агрегує списки ріелторів, клієнтів та нерухомості, надає методи для додавання, редагування та видалення записів, а також реалізує запити та

статистику (середня вартість, найбільша кількість кімнат, найпопулярніша площа, об’єкти з найбільшим попитом).

Для опису спільних характеристик використано перерахування:

- **ServiceType** — визначає тип послуги (купівля, оренда, консультація).
- **AvailabilityStatus** — визначає статус нерухомості (на продаж, продана, оренда).

Графічний інтерфейс реалізовано на базі **Java Swing**. Основний клас **RealEstateApp** відповідає за запуск програми та створення головного вікна. Інтерфейс побудований у вигляді **JTabbedPane** з трьома вкладками: «Ріелтор», «Клієнт» та «Нерухомість». Кожна вкладка містить таблицю даних та панель керування з кнопками CRUD-операцій. Додатково реалізовано діалоги для введення та редагування даних, а також кнопки для виконання запитів і збереження інформації у файл CSV.

Таким чином, структура програми поєднує об’єктно-орієнтовану модель даних із графічним інтерфейсом, що забезпечує зручну роботу користувача та автоматизацію обліку у ріелторській фірмі.

5. UML-діаграми

5.1. Діаграма варіантів використання (Use Case Diagram)

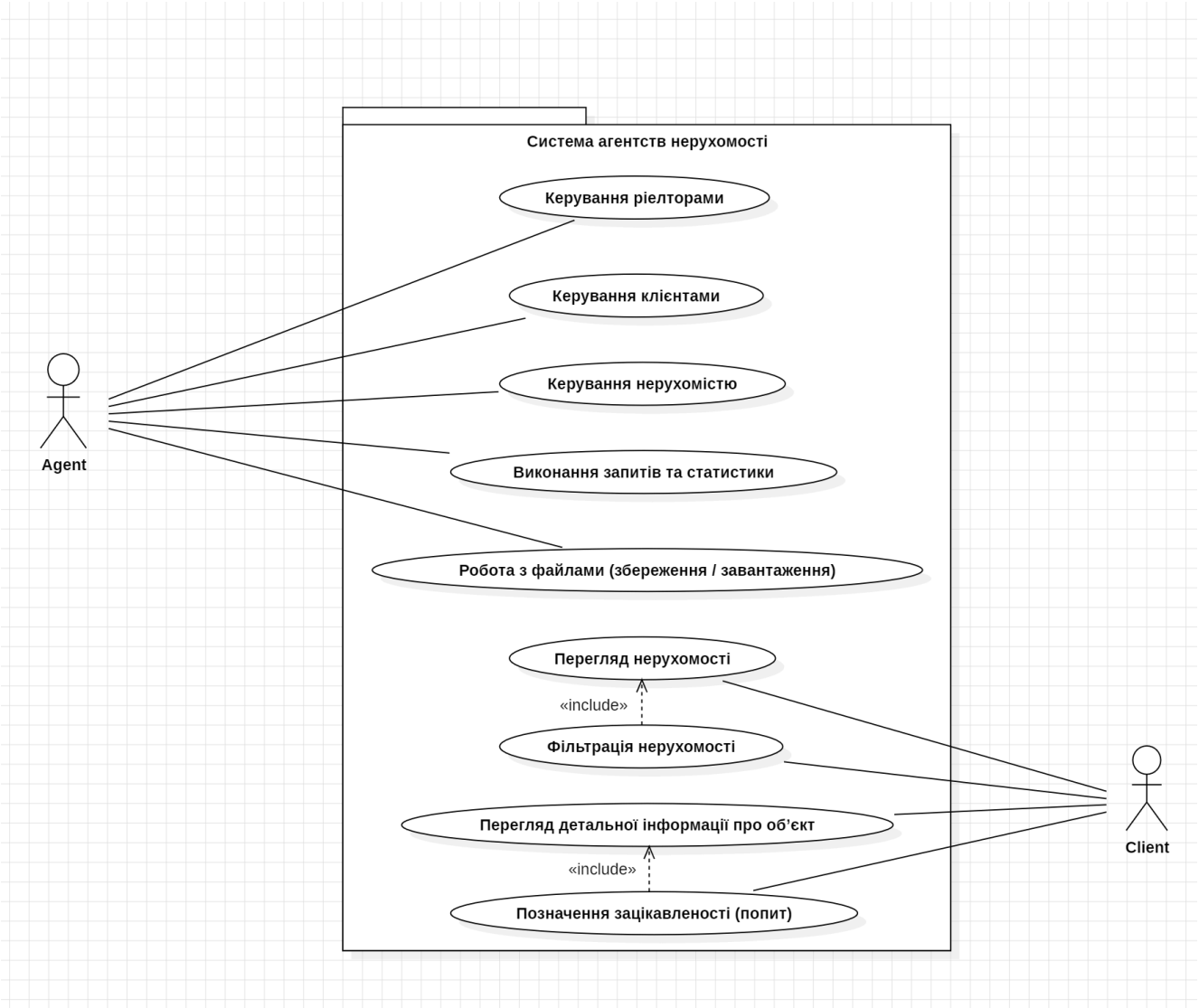


Рисунок 5.1 – Діаграма варіантів використання системи ріелторської фірми

На діаграмі зображено взаємодію двох основних акторів — Agent (працівник агентства) та Client (клієнт) — із функціональністю системи.

Agent має доступ до операцій керування ріелторами, клієнтами та об'єктами нерухомості, а також може виконувати запити, переглядати статистику та працювати з файлами.

Client може переглядати нерухомість, фільтрувати об'єкти за параметрами, переглядати детальну інформацію та позначати зацікавленість.

Зв'язки типу <<Include>> вказують на залежність між діями:

- фільтрація є частиною перегляду нерухомості

- позначення зацікавленості — частина перегляду детальної інформації
- окремі запити — частина загального кейсу «Виконання запитів та статистики»

5.2. Діаграма класів (Class Diagram)

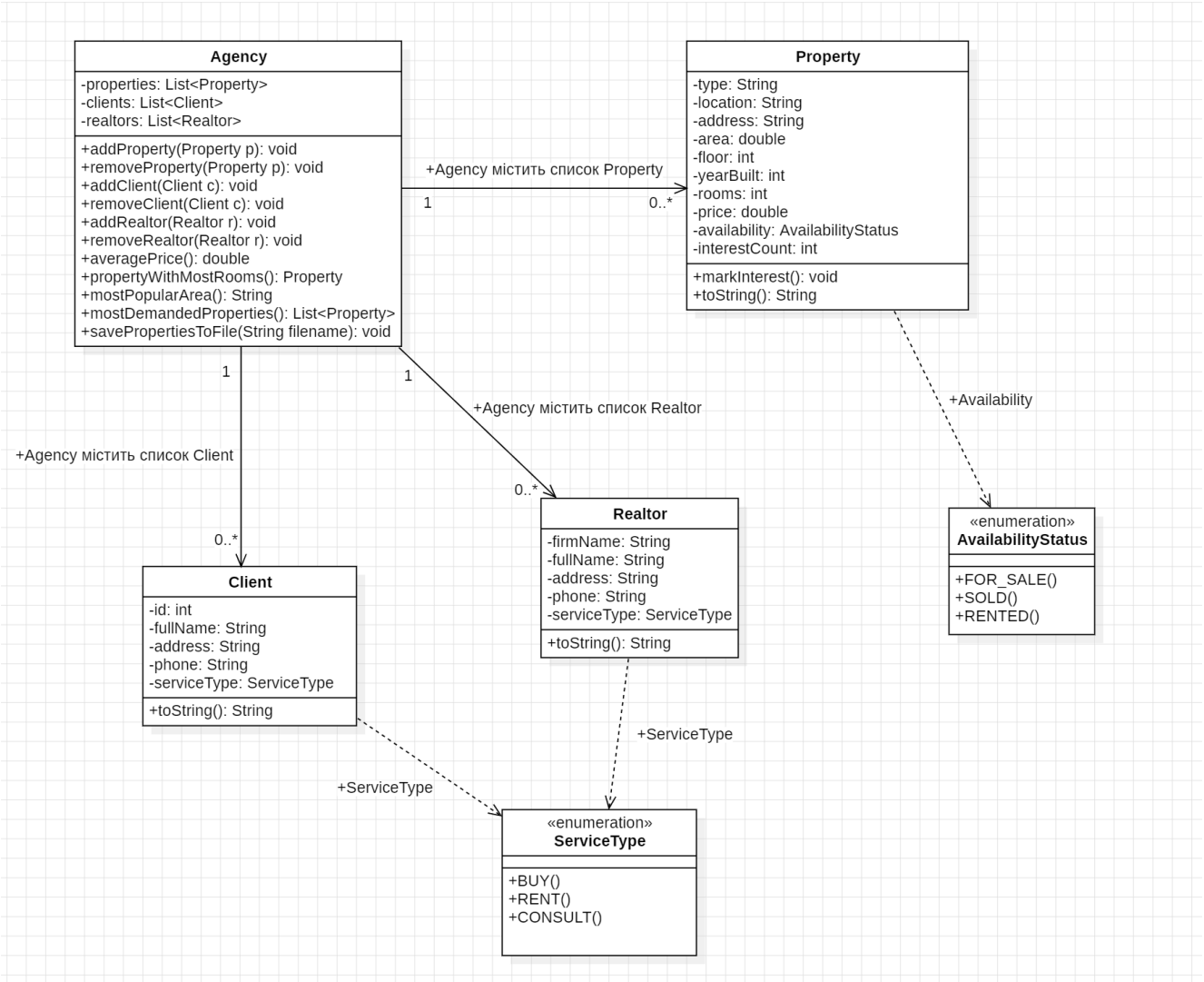


Рисунок 5.2 – Діаграма класів системи агентства нерухомості

На діаграмі зображено структуру основних класів, що реалізують функціональність системи.

Центральним елементом є клас Agency, який агрегує списки об’єктів нерухомості (Property), клієнтів (Client) та ріелторів (Realtor).

Клас Property містить атрибути, що описують нерухомість: тип, локацію, адресу, площу, поверх, рік побудови, кількість кімнат, вартість, статус та попит. Статус визначається через перерахування AvailabilityStatus.

Класи Client та Realtor містять інформацію про ПІБ, адресу, телефон та тип послуги, що визначається через перерахування ServiceType.

Усі зв’язки між класами супроводжуються мультиплікаторами та ролями, що відображають типи асоціацій:

- агрегація між Agency та сутностями
- залежність між класами та перерахуваннями

5.3. Діаграма послідовностей (Sequence Diagram)

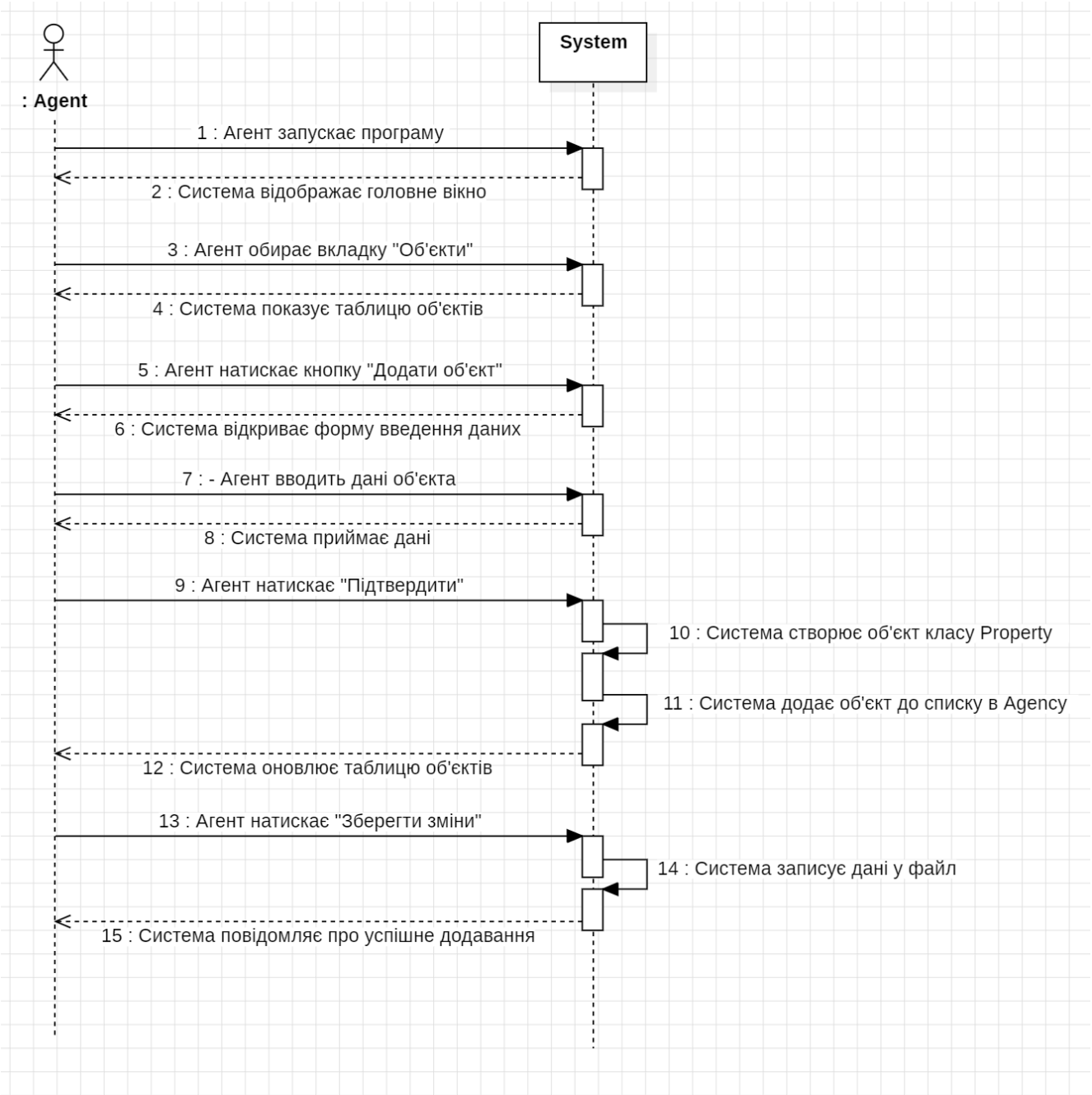


Рисунок 5.3 – Діаграма послідовностей процесу додавання об'єкта нерухомості

На діаграмі послідовностей зображено покрокову взаємодію між користувачем системи (Agent) та програмною системою агентства нерухомості під час додавання нового об'єкта до бази даних. Діаграма демонструє логіку роботи графічного інтерфейсу, обробку введених даних, створення об'єкта класу Property, оновлення внутрішніх структур у класі Agency та збереження інформації у зовнішній файл.

Процес починається з запуску програми, після чого система ініціалізує головне вікно. Користувач обирає вкладку «Об'єкти», і система відображає таблицю з наявними об'єктами нерухомості. Далі агент натискає кнопку «Додати об'єкт», що викликає відкриття форми введення.

Агент заповнює форму, вводячи тип нерухомості, адресу, площу, кількість кімнат, поверх, рік побудови, ціну, статус та локацію. Після підтвердження введених даних система створює новий об'єкт класу Property, додає його до списку у класі Agency та оновлює таблицю на екрані.

На завершальному етапі агент натискає кнопку «Зберегти зміни», після чого система записує оновлені дані у зовнішній файл (наприклад, properties_data.csv). Після успішного завершення операції система повідомляє користувача про те, що об'єкт було успішно додано.

Кожна дія на діаграмі представлена у вигляді синхронного повідомлення між учасниками (Agent і System), а внутрішні дії системи (створення об'єкта, додавання до списку, запис у файл) показані як самовиклики (self-call) на лінії життя системи. Такий підхід дозволяє чітко відобразити як зовнішню взаємодію, так і внутрішню логіку обробки даних.

- **Висновок:** Таким чином, UML-діаграми відображають основні аспекти роботи системи: взаємодію користувачів із функціональністю, структуру класів та послідовність виконання ключових процесів.

6. Реалізація програми на Java

6.1. Опис середовища розробки

Для реалізації курсової роботи було використано сучасне середовище розробки, яке забезпечує зручну роботу з графічними інтерфейсами, об’єктно-орієнтованим програмуванням та збереженням даних.

Мова програмування:

Java — об’єктно-орієнтована мова програмування, яка дозволяє створювати кросплатформні додатки. Вона має потужну стандартну бібліотеку, підтримує роботу з графічним інтерфейсом через бібліотеку Swing, а також забезпечує зручну роботу з файлами та колекціями.

Версія JDK:

Проєкт реалізовано з використанням Java Development Kit версії 17, яка підтримує всі необхідні функції для побудови GUI-додатків.

Середовище розробки (IDE):

Для написання та тестування програми використовувалося середовище IntelliJ IDEA. IDE забезпечує зручну навігацію по класах, автоматичне форматування коду, підказки та запуск програми в один клік.

Операційна система:

Розробка здійснювалася на Windows 11, яка забезпечує стабільну роботу середовища Java та підтримку графічних компонентів.

Структура проєкту:

Проєкт складається з кількох класів, кожен з яких відповідає за свою частину логіки:

- **Property:** модель об’єкта нерухомості
- **Client:** модель клієнта агентства
- **Realtor:** модель ріелтора
- **Agency:** логіка зберігання та обробки даних (списки об’єктів, клієнтів, ріелторів, запити, статистика, збереження у файл)
- **RealEstateApp:** головний клас із графічним інтерфейсом (GUI)

Використані бібліотеки:

- **javax.swing*** — базова бібліотека для створення графічного інтерфейсу (вікна, панелі, кнопки, таблиці, вкладки, діалогові повідомлення).
- **javax.swing.table.DefaultTableModel** — модель даних для таблиць, яка дозволяє додавати, редагувати та видаляти рядки у графічному інтерфейсі.
- **java.awt.BorderLayout** — менеджер розташування компонентів у вікні, що забезпечує поділ на області (північ, південь, схід, захід, центр).
- **java.awt.GridLayout** — менеджер розташування, який розміщує елементи у вигляді таблиці з рядками та стовпцями.
- **java.awt.event*** — обробка подій користувача (натискання кнопок, введення з клавіатури, взаємодія з мишею).
- **java.io.FileWriter** — запис текстових даних у файл.
- **java.io.IOException** — обробка виняткових ситуацій, що виникають під час роботи з файлами.
- **java.io.PrintWriter** — зручний інструмент для форматованого запису тексту у файл.
- **java.util*** — робота з колекціями (List, ArrayList, HashMap), утиліти для сортування та пошуку, генерація випадкових значень, тощо.

Спосіб запуску програми:

Запуск здійснюється через метод main() у класі RealEstateApp. Програма відкриває графічне вікно, в якому користувач може додавати, редагувати, переглядати та зберігати об’єкти нерухомості, клієнтів і ріелторів, а також виконувати запити та переглядати статистику.

6.2. Структура та взаємодія класів

Програма реалізована у вигляді набору взаємопов’язаних класів, кожен з яких виконує окрему функцію в межах системи управління об’єктами нерухомості. Нижче — оновлена версія відповідно до нашого коду: прибрано Deal, додано Realtor та перерахування, розширено атрибути Property, узгоджено методи Agency і взаємодію GUI.

- **Клас Property — модель об’єкта нерухомості**

Клас Property описує всі характеристики нерухомості: тип, локація, адреса, площа, поверх, рік побудови, кількість кімнат, ціна, статус доступності та поле «попиту». Реалізує інтерфейси Printable та Filterable, що дозволяє виводити інформацію та здійснювати пошук за критеріями.

```
// Клас Нерухомість: дані про об'єкт + поле попиту
class Property implements Printable, Filterable {
   private double area;
   private int floor;
   private int rooms;
   private int yearBuilt;
   private double price;
   private AvailabilityStatus availability;
   private String address;
   private String location; // для запиту "популярна локація"
   private String type;     // тип нерухомості (квартира/будинок
тощо)
   private int interestCount = 0; // "попит": збільшується при
зацікавленні клієнта

   public Property(String type, String location, String address,
double price, AvailabilityStatus availability,
                   double area, int floor, int yearBuilt, int rooms)
{
       this.type = type;
       this.location = location;
       this.address = address;
       this.price = price;
       this.availability = availability;
       this.area = area;
       this.floor = floor;
       this.yearBuilt = yearBuilt;
       this.rooms = rooms;
   }

   // Гетери/сетери
   public String getType() { return type; }
   public String getLocation() { return location; }
   public String getAddress() { return address; }
   public double getPrice() { return price; }
   public AvailabilityStatus getAvailability() { return
availability; }
   public double getArea() { return area; }
   public int getFloor() { return floor; }
   public int getYearBuilt() { return yearBuilt; }
   public int getRooms() { return rooms; }
   public int getInterestCount() { return interestCount; }

   public void setType(String v) { type = v; }
   public void setLocation(String v) { location = v; }
   public void setAddress(String v) { address = v; }
```

```
    public void setPrice(double v) { price = v; }
    public void setAvailability(AvailabilityStatus v) { availability
= v; }
    public void setArea(double v) { area = v; }
    public void setFloor(int v) { floor = v; }
    public void setYearBuilt(int v) { yearBuilt = v; }
    public void setRooms(int v) { rooms = v; }

    // Метод для позначення попиту
    public void markInterest() { interestCount++; }

    @Override
    public boolean matches(FilterCriteria c) {
        if (c == null) return true;
        boolean okType = c.type == null || c.type.isEmpty() ||
type.equalsIgnoreCase(c.type);
        boolean okLoc  = c.location == null || c.location.isEmpty()
|| location.equalsIgnoreCase(c.location);
        boolean okPrice = (c.minPrice <= 0 || price >= c.minPrice) &&
(c.maxPrice <= 0 || price <= c.maxPrice);
        boolean okStatus = c.status == null || availability ==
c.status;
        return okType && okLoc && okPrice && okStatus;
    }

    @Override
    public void print() {
        System.out.println(type + ", " + location + ", " + address +
", " + price + ", " + availability);
    }
}
```

- **Клас Agency — логіка зберігання та обробки об’єктів**

Клас Agency відповідає за управління даними агентства нерухомості. Він містить списки об’єктів (Property), клієнтів (Client) та ріелторів (Realtor). Реалізує CRUD-операції (додавання, видалення), збереження даних у файл у форматі CSV, а також методи для виконання статистичних запитів: середня ціна, найпопулярніша локація, об’єкт із найбільшою кількістю кімнат, адреси об’єктів на продаж та список найбільш затребуваних об’єктів.

```
// Менеджер даних: списки і запити/статистика
class Agency {
    private String name;

    List<Property> properties = new ArrayList<>();
    List<Client> clients = new ArrayList<>();
    List<Realtor> realtors = new ArrayList<>();

    public Agency(String name) { this.name = name; }
```

```
    public String getName() { return name; }

    // CRUD для нерухомості
    public void addProperty(Property p) { properties.add(p); }
    public void removeProperty(int index) { if (index >= 0 && index <
properties.size()) properties.remove(index); }

    // CRUD для клієнтів
    public void addClient(Client c) { clients.add(c); }
    public void removeClient(int index) { if (index >= 0 && index <
clients.size()) clients.remove(index); }

    // CRUD для рієлторів
    public void addRealtor(Realtor r) { realtors.add(r); }
    public void removeRealtor(int index) { if (index >= 0 && index <
realtors.size()) realtors.remove(index); }

    // Збереження нерухомості у CSV
    public void savePropertiesToFile(String filename) {
        try (PrintWriter writer = new PrintWriter(new
FileWriter(filename))) {
            for (Property p : properties) {
                writer.println(p.getType() + "," + p.getLocation() +
"," + p.getAddress() + "," + p.getPrice() + "," +
                        p.getAvailability() + "," + p.getArea() + ","
+ p.getFloor() + "," + p.getYearBuilt() + "," +
                        p.getRooms() + "," + p.getInterestCount());
            }
            JOptionPane.showMessageDialog(null, "Дані нерухомості
збережено у файл: " + filename);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(null, "Помилка запису у
файл.");
        }
    }

    // Запити/статистика
    public double averagePrice() {
        if (properties.isEmpty()) return 0;
        double sum = 0;
        for (Property p : properties) sum += p.getPrice();
        return sum / properties.size();
    }

    public String mostPopularArea() {
        Map<String, Integer> count = new HashMap<>();
        for (Property p : properties) {
            count.put(p.getLocation(),
count.getOrDefault(p.getLocation(), 0) + 1);
        }
        return count.isEmpty() ? "Немає даних"
                : Collections.max(count.entrySet(),
Map.Entry.comparingByValue()).getKey();
```

```
    }

    public Property propertyWithMostRooms() {
        if (properties.isEmpty()) return null;
        return Collections.max(properties,
Comparator.comparingInt(Property::getRooms));
    }

    public List<String> saleAddresses() {
        List<String> result = new ArrayList<>();
        for (Property p : properties) {
            if (p.getAvailability() == AvailabilityStatus.НА_ПРОДАЖ)
{
                result.add(p.getAddress());
            }
        }
        return result;
    }

    public List<Property> mostDemandedProperties() {
        List<Property> copy = new ArrayList<>(properties);

copy.sort(Comparator.comparingInt(Property::getInterestCount).revers
ed());
        return copy;
    }
}
```

- **Клас Client — модель клієнта агентства**

Клас Client зберігає дані про клієнта агентства: повне ім’я, адресу, телефон та тип послуги (купівля, оренда чи консультація). Реалізує інтерфейс Printable, що дозволяє виводити інформацію про клієнта у зручному форматі..

```
// Клас Клієнт: дані про клієнта агентства
class Client implements Printable {
    private String fullName;
    private String address;
    private String phone;
    private ServiceType serviceType;

    public Client(String fullName, String address, String phone,
ServiceType serviceType) {
        this.fullName = fullName;
        this.address = address;
        this.phone = phone;
        this.serviceType = serviceType;
    }

    // Гетери/сетери
    public String getFullName() { return fullName; }
    public String getAddress() { return address; }
```

```
    public String getPhone() { return phone; }
    public ServiceType getServiceType() { return serviceType; }

    public void setFullName(String v) { fullName = v; }
    public void setAddress(String v) { address = v; }
    public void setPhone(String v) { phone = v; }
    public void setServiceType(ServiceType v) { serviceType = v; }

    @Override
    public void print() {
        System.out.println(fullName + " | " + address + " | " + phone
+ " | " + serviceType);
    }
}
```

- **Клас Realtor — модель рієлтора агентства**

Клас Realtor зберігає дані про співробітника агентства: назву фірми, повне ім’я, адресу, телефон та тип послуги (купівля, оренда чи консультація). Реалізує інтерфейс Printable, що дозволяє виводити інформацію про рієлтора у зручному форматі.

```
// Клас Рієлтор: дані про співробітника агентства
class Realtor implements Printable {
    private String firmName;
    private String fullName;
    private String address;
    private String phone;
    private ServiceType serviceType;

    public Realtor(String firmName, String fullName, String address,
String phone, ServiceType serviceType) {
        this.firmName = firmName;
        this.fullName = fullName;
        this.address = address;
        this.phone = phone;
        this.serviceType = serviceType;
    }
    // Гетери/сетери
    public String getFirmName() { return firmName; }
    public String getFullName() { return fullName; }
    public String getAddress() { return address; }
    public String getPhone() { return phone; }
    public ServiceType getServiceType() { return serviceType; }

    public void setFirmName(String v) { firmName = v; }
    public void setFullName(String v) { fullName = v; }
    public void setAddress(String v) { address = v; }
    public void setPhone(String v) { phone = v; }
    public void setServiceType(ServiceType v) { serviceType = v; }
```

```
    @Override
    public void print() {
        System.out.println(firmName + " | " + fullName + " | " +
address + " | " + phone + " | " + serviceType);
    }
}
```

- **Клас RealEstateApp — графічний інтерфейс користувача**

Клас RealEstateApp відповідає за створення графічного інтерфейсу програми агентства нерухомості. Він формує головне вікно, вкладки, таблиці та кнопки, а також забезпечує взаємодію користувача з даними через клас Agency. У цьому класі реалізовано форми для додавання та редагування об’єктів, а також обробку подій, що дозволяє виконувати CRUD-операції та зберігати дані у файл.

```
// Головний клас з графічним інтерфейсом
public class RealEstateApp {
    private Agency agency = new Agency("Нерухомість Плюс");
    private javax.swing.table.DefaultTableModel tableModel;

    public RealEstateApp() {
        // Створення GUI: головне вікно, вкладки "Нерухомість",
"Клієнти", "Рієлтори"
        // Ініціалізація таблиць, кнопок "Додати", "Редагувати",
"Видалити", "Зберегти"
        // Підключення обробників подій до методів Agency
    }

    private void openAddDialog() {
        // Форма додавання нового об'єкта:
        // введення типу, адреси, локації, площі, кімнат, поверху,
року побудови, ціни, статусу
        // Підтвердження -> agency.addProperty(...)
        // Оновлення таблиці
    }

    private void openEditDialog(int row) {
        // Форма редагування:
        // оновлення полів обраного Property
        // Збереження змін і оновлення таблиці
    }

    public static void main(String[] args) {
        // Запуск програми: створення екземпляра RealEstateApp
        // Відображення головного вікна
    }
}
```

Взаємозв'язки між класами:

- **RealEstateApp** — головний клас, який створює графічний інтерфейс користувача та забезпечує взаємодію з даними через клас Agency.
- **Agency** — менеджер даних, що зберігає списки об'єктів (Property), клієнтів (Client) та рієлторів (Realtor), а також надає методи для їхньої обробки, виконання CRUD-операцій, статистики та збереження у файл.
- **Property** — основна модель даних, яка описує характеристики нерухомості, підтримує редагування, пошук за критеріями та збереження у файл.
- **Client** — модель клієнта агентства, що містить персональні дані та тип послуги, використовується для обліку клієнтів.
- **Realtor** — модель рієлтора, що містить інформацію про співробітників агентства та їхні послуги.
- **AvailabilityStatus** — перерахування, яке визначає статус нерухомості (на продаж, продано, орендовано), використовується у класі Property.
- **ServiceType** — перерахування, яке визначає тип послуги (купівля, оренда, консультація), використовується у класах Client та Realtor.

6.3. Графічний інтерфейс та основні операції (три вкладки).

Графічний інтерфейс програми реалізовано у класі RealEstateApp за допомогою бібліотеки Swing. Інтерфейс дозволяє користувачу керувати даними агентства нерухомості через три вкладки (JTabbedPane):

- «Рієлтор» — управління співробітниками агентства
- «Клієнт» — управління клієнтами
- «Нерухомість» — управління об'єктами нерухомості

Кожна вкладка містить таблицю (JTable) для відображення даних та панель з кнопками CRUD-операцій (додавання, редагування, видалення). Додатково реалізовано меню «Запити» для перегляду статистики та списків.

- **Головне вікно**

Головне вікно створюється у конструкторі класу. Воно містить три вкладки та меню запитів:

```
        frame = new JFrame("Агентство нерухомості — " +
agency.getName());
```

```
        frame.setSize(1000, 600);

        JTabbedPane tabs = new JTabbedPane();
tabs.addTab("Рієлтор", buildRealtorPanel());
        tabs.addTab("Клієнт", buildClientPanel());
        tabs.addTab("Нерухомість", buildPropertyPanel());

        frame.add(tabs, BorderLayout.CENTER);
frame.setVisible(true);
```

- **Вкладка «Рієлтор»**
    - Таблиця з даними про рієлторів: назва фірми, ПІБ, адреса, телефон, тип послуги.
    - Кнопки: «Додати рієлтора», «Редагувати», «Видалити».
    - Діалог (JDialog) для введення/редагування даних рієлтора.

```
        realtorModel = new DefaultTableModel(new Object[]{
   "Назва фірми", "ПІБ", "Адреса", "Телефон", "Тип послуги"
}, 0);
        realtorTable = new JTable(realtorModel);
```

- **Вкладка «Нерухомість»**
    - Таблиця з даними про об’єкти: тип, локація, адреса, ціна, статус, площа, поверх, рік побудови, кількість кімнат, попит.
    - Кнопки: «Додати об’єкт», «Редагувати», «Видалити», «Зберегти у файл (CSV)».
    - Діалог (JDialog) для введення/редагування даних нерухомості.
    - Збереження у файл виконується методом savePropertiesToFile() класу Agency.

```
        propertyModel = new DefaultTableModel(new Object[]{
   "Тип", "Локація", "Адреса", "Ціна", "Статус", "Площа",
"Поверх", "Рік", "Кімнат", "Попит"
}, 0);
        propertyTable = new JTable(propertyModel);
```

- **Меню «Запити»**

У головному вікні програми реалізовано меню JMenuBar з пунктами для виконання запитів:

- «Список рієлторів» — виводить усіх рієлторів агентства.
- «Список клієнтів» — виводить усіх клієнтів агентства.

- «Список адрес продажу» — показує адреси об’єктів зі статусом «НА_ПРОДАЖ».
- «Нерухомість з найбільшою кількістю кімнат» — викликає метод propertyWithMostRooms().
- «Середня вартість нерухомості» — викликає метод averagePrice().
- «Об’єкти з найбільшим попитом» — викликає метод mostDemandedProperties().

6.4. Опис додаткових можливостей програми

Окрім базових CRUD-операцій та роботи з таблицями, програма має низку додаткових можливостей, які розширюють функціонал агентства нерухомості. Вони реалізовані у класах Property, Agency та RealEstateApp.

1. Інтерфейси Printable та Filterable

- **Printable** — дозволяє кожному класу (Property, Client, Realtor) виводити свої дані у консоль у зручному форматі.
- **Filterable** — використовується у класі Property для перевірки відповідності об’єкта критеріям пошуку (тип, локація, ціна, статус).

```
interface Printable { void print(); }
interface Filterable { boolean matches(FilterCriteria criteria); }
```

2. Критерії фільтрації

Клас **FilterCriteria** дозволяє задавати умови для пошуку нерухомості: тип, локація, мінімальна та максимальна ціна, статус доступності.

```
class FilterCriteria {
   String type, location;
   double minPrice, maxPrice;
   AvailabilityStatus status;
   // Конструктор для ініціалізації критеріїв
}
```

3. Поле «попит» у класі Property

Кожен об’єкт нерухомості має поле interestCount, яке збільшується при позначенні зацікавлення клієнтом. Це дозволяє визначати найбільш затребувані об’єкти.

```
public void markInterest() { interestCount++; }
```

4. Позначення зацікавлення клієнтом

У вкладці «Клієнт» є кнопка «Позначити зацікавлення нерухомістю». Вона відкриває діалог, де клієнт може вибрати об’єкт нерухомості. Після вибору викликається метод markInterest().

```
        String choice = (String) JOptionPane.showInputDialog(frame,
"Оберіть об'єкт:",
                "Зацікавлення", JOptionPane.PLAIN_MESSAGE, null,
items, items[0]);
if (choice != null) {
        int idx = Arrays.asList(items).indexOf(choice);
    agency.properties.get(idx).markInterest();
}
```

5. Меню запитів

У головному вікні реалізовано меню «Запити», яке дозволяє швидко отримати статистику та списки:

- Список рієлторів
- Список клієнтів
- Список адрес продажу
- Нерухомість з найбільшою кількістю кімнат
- Середня вартість нерухомості
- Об’єкти з найбільшим попитом

```
        JMenuItem miAvgPrice = new JMenuItem("Середня вартість
нерухомості");
miAvgPrice.addActionListener(e -> showAveragePrice());
```

6. Статистичні методи у класі Agency

- averagePrice() — обчислює середню ціну нерухомості.
- mostPopularArea() — визначає найпопулярнішу локацію.
- propertyWithMostRooms() — знаходить об’єкт із найбільшою кількістю кімнат.
- saleAddresses() — повертає список адрес об’єктів зі статусом «НА_ПРОДАЖ».
- mostDemandedProperties() — сортує об’єкти за рівнем попиту.

7. Тестування програми

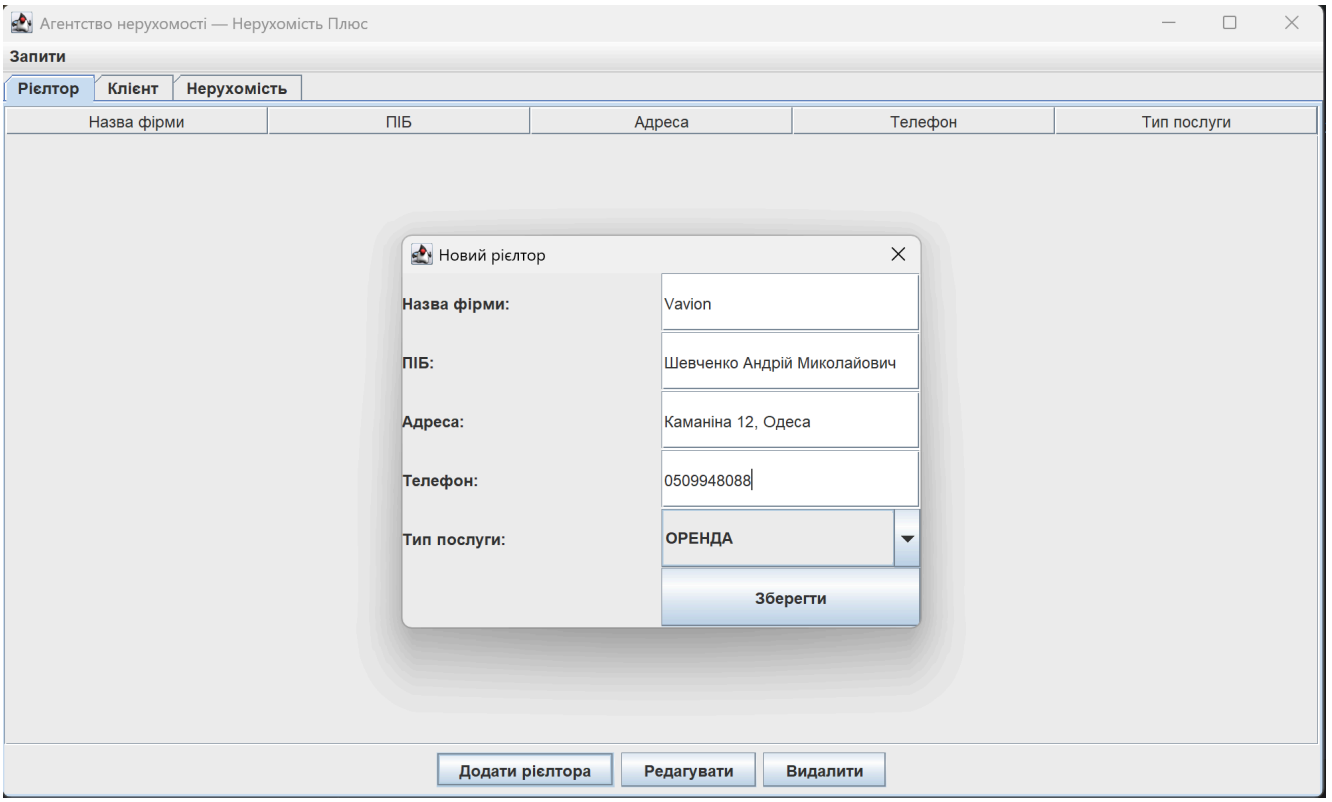
Інструкція користувача

7.1. CRUD-операції для ріелтора

У цьому підпункті демонструється робота з вкладкою «Ріелтор». Для перевірки функціоналу виконуються всі базові операції CRUD (Create, Read, Update, Delete). Кожна операція супроводжується скріншотом та поясненням.

1. Додавання ріелтора (Create)

- **Опис:** при натисканні кнопки «Додати ріелтора» відкривається діалогове вікно (JDialog) з полями для введення даних: назва фірми, ПІБ, адреса, телефон, тип послуги.
- **Дії користувача:** заповнити всі поля та натиснути «Зберегти».
- **Результат:** новий ріелтор додається до списку агентства (Agency) та відображається у таблиці.

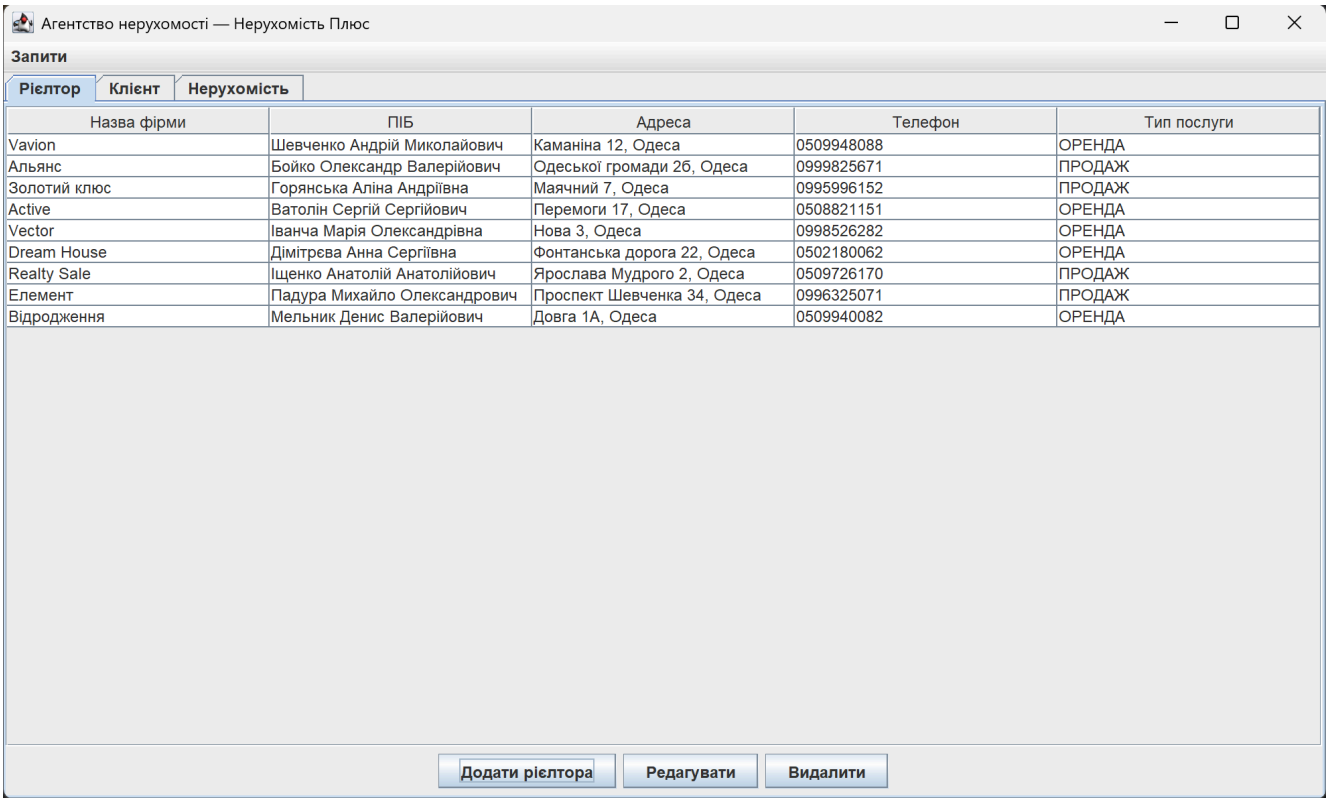


Скріншот 7.1.1 — форма додавання нового ріелтора.

2. Перегляд списку рієлторів (Read)

- **Опис:** таблиця у вкладці «Ріелтор» відображає всі додані записи.

- **Дії користувача:** після додавання кількох рієлторів можна переглянути повний список.
- **Результат:** користувач бачить усіх рієлторів агентства у вигляді рядків таблиці.

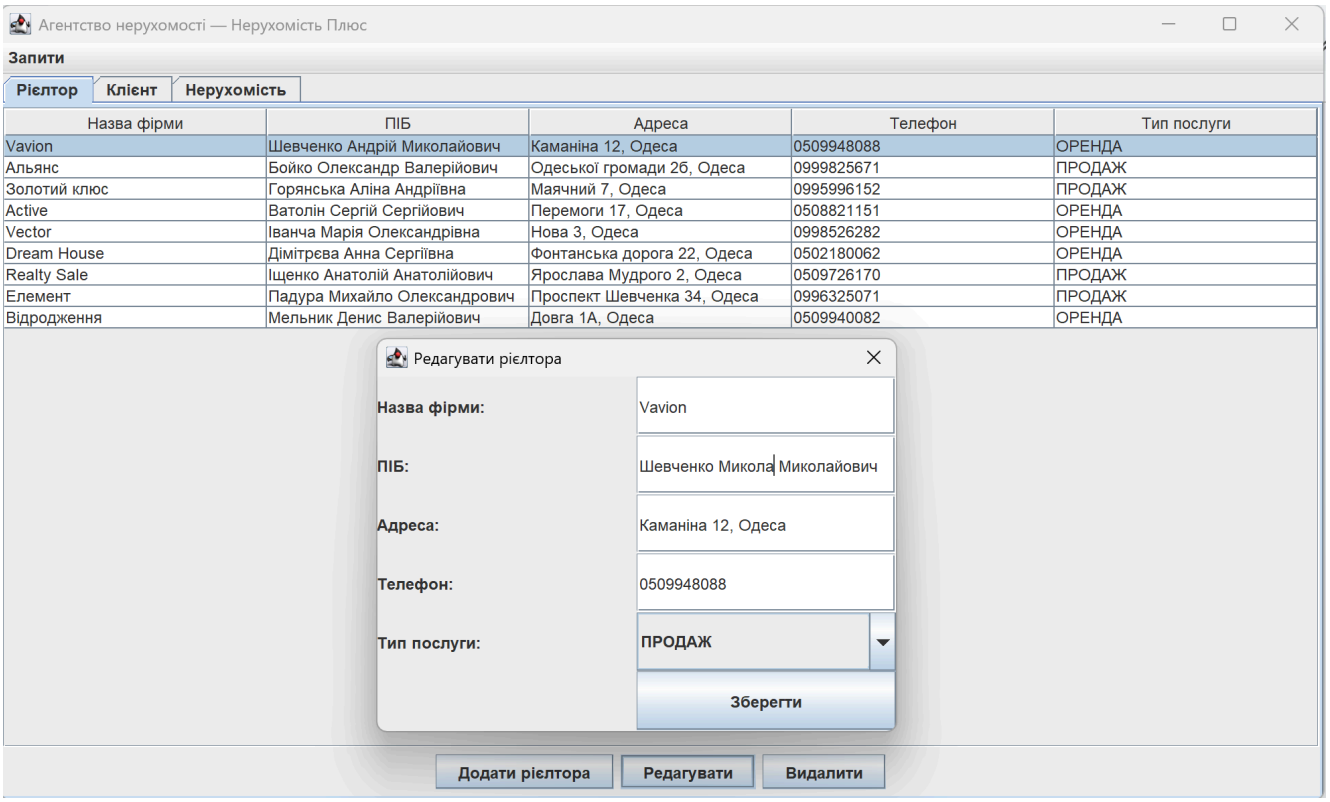


Назва фірми	ПІБ	Адреса	Телефон	Тип послуги
Vavion	Шевченко Андрій Миколайович	Каманіна 12, Одеса	0509948088	ОРЕНДА
Альянс	Бойко Олександр Валерійович	Одеської громади 2б, Одеса	0999825671	ПРОДАЖ
Золотий клюс	Горянська Аліна Андріївна	Маячний 7, Одеса	0995996152	ПРОДАЖ
Active	Ватолін Сергій Сергійович	Перемоги 17, Одеса	0508821151	ОРЕНДА
Vector	Іванча Марія Олександрівна	Нова 3, Одеса	0998526282	ОРЕНДА
Dream House	Дімітрєва Анна Сергіївна	Фонтанська дорога 22, Одеса	0502180062	ОРЕНДА
Realty Sale	Іщенко Анатолій Анатолійович	Ярослава Мудрого 2, Одеса	0509726170	ПРОДАЖ
Елемент	Падура Михайло Олександрович	Проспект Шевченка 34, Одеса	0996325071	ПРОДАЖ
Відродження	Мельник Денис Валерійович	Довга 1А, Одеса	0509940082	ОРЕНДА

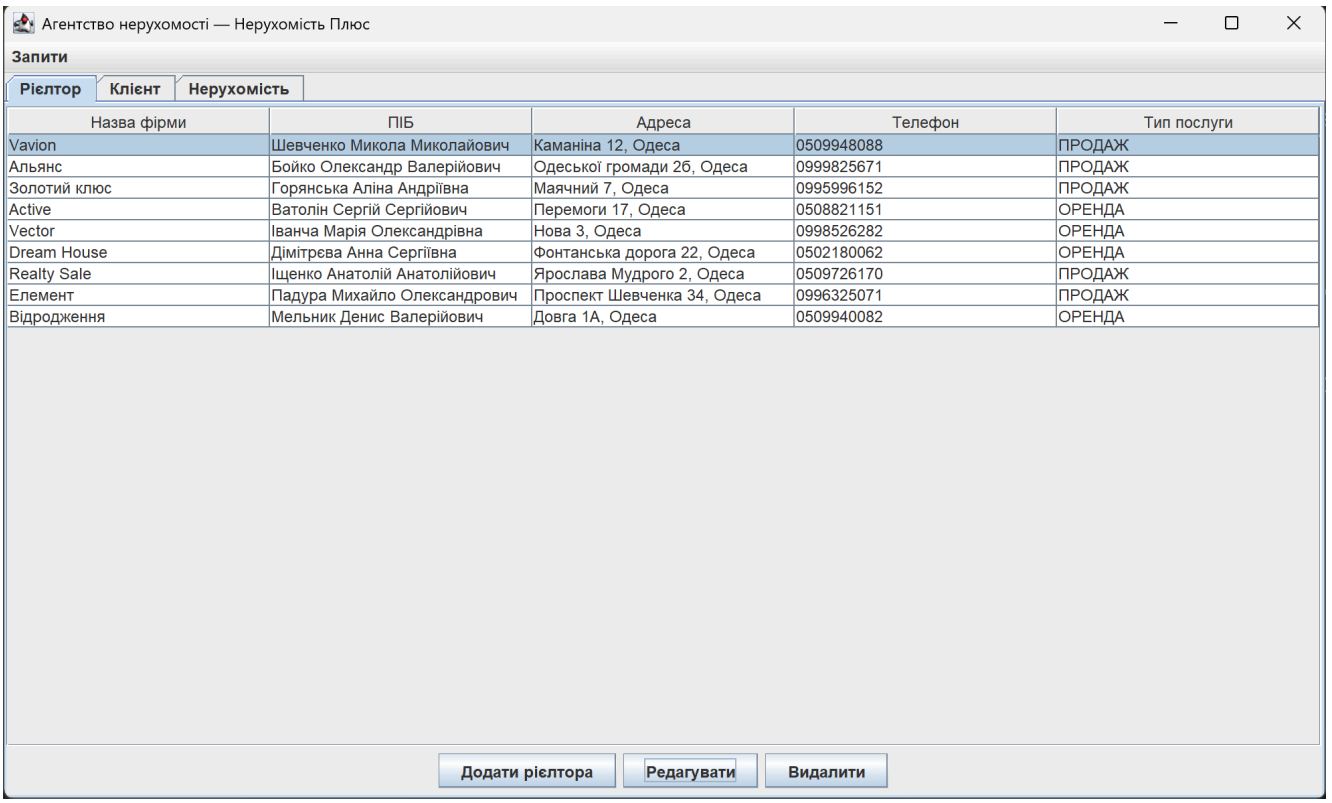
Скріншот 7.1.2 — таблиця зі списком кількох рієлторів

3. Редагування рієлтора (Update)

- **Опис:** при натисканні кнопки «Редагувати» відкривається форма з уже заповненими даними обраного рієлтора.
- **Дії користувача:** змінити потрібні поля (наприклад, телефон чи тип послуги) та натиснути «Зберегти».
- **Результат:** дані у таблиці оновлюються відповідно до змін.



Скріншот 7.1.3 — форма редагування рієлтора.

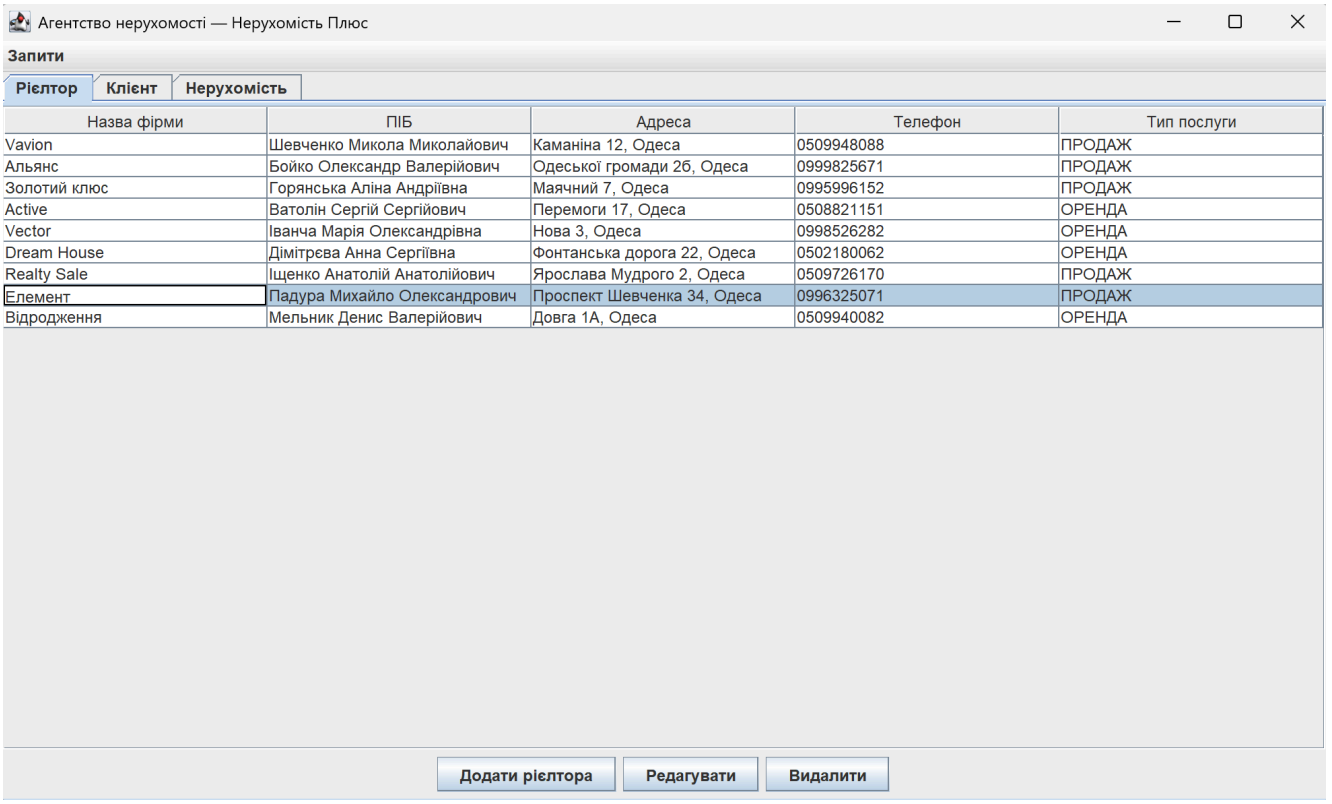


Скріншот 7.1.4 — таблиця після редагування

4. Видалення рієлтора (Delete)

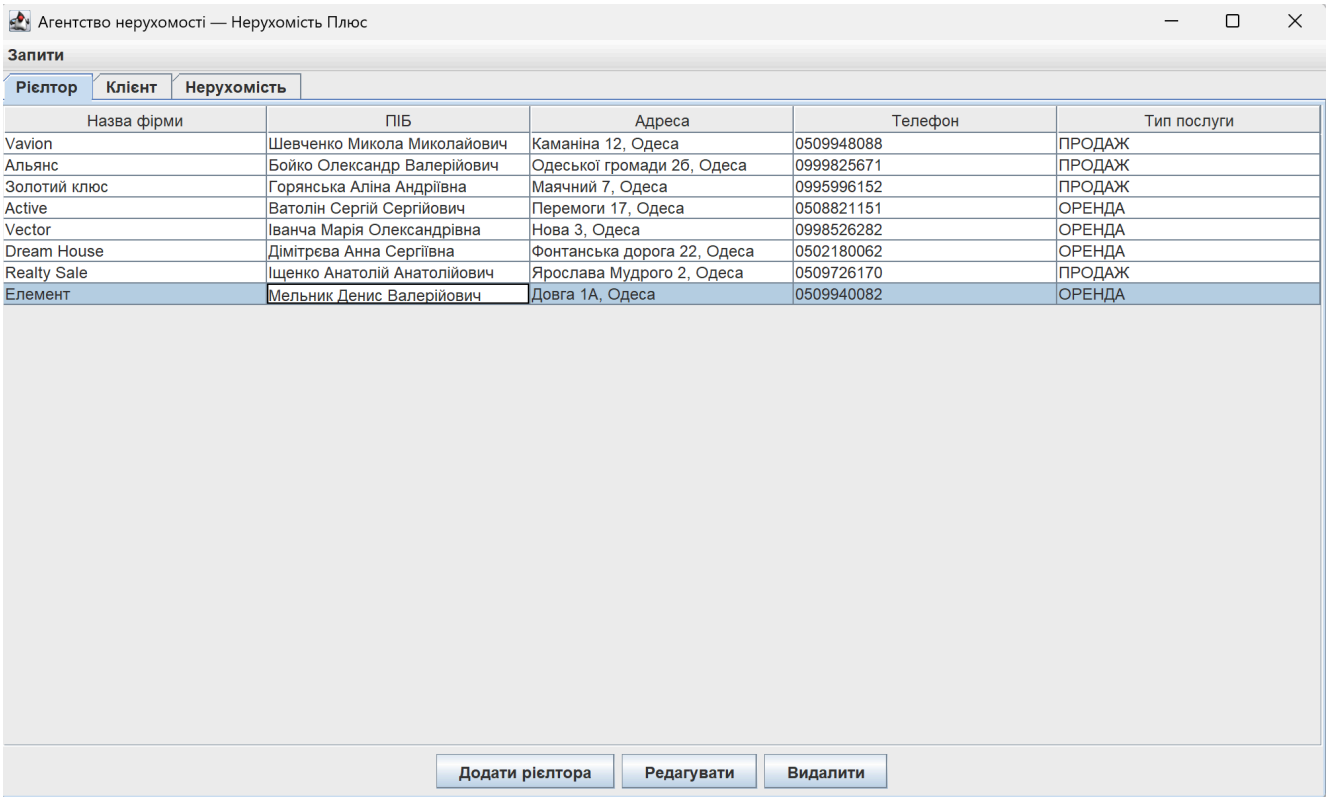
- **Опис:** при натисканні кнопки «Видалити» обраний рієлтор видаляється зі списку агентства та з таблиці.

- **Дії користувача:** вибрати рядок у таблиці та натиснути «Видалити».
- **Результат:** відповідний рядок зникає з таблиці, а об’єкт видаляється зі списку у класі Agency.



Назва фірми	ПІБ	Адреса	Телефон	Тип послуги
Vavion	Шевченко Микола Миколайович	Каманіна 12, Одеса	0509948088	ПРОДАЖ
Альянс	Бойко Олександр Валерійович	Одеської громади 2б, Одеса	0999825671	ПРОДАЖ
Золотий клюс	Горянська Аліна Андріївна	Маячний 7, Одеса	0995996152	ПРОДАЖ
Active	Ватолін Сергій Сергійович	Перемоги 17, Одеса	0508821151	ОРЕНДА
Vector	Іванча Марія Олександрівна	Нова 3, Одеса	0998526282	ОРЕНДА
Dream House	Дімітрєва Анна Сергіївна	Фонтанська дорога 22, Одеса	0502180062	ОРЕНДА
Realty Sale	Іщенко Анатолій Анатолійович	Ярослава Мудрого 2, Одеса	0509726170	ПРОДАЖ
Елемент	Падура Михайло Олександрович	Проспект Шевченка 34, Одеса	0996325071	ПРОДАЖ
Відродження	Мельник Денис Валерійович	Довга 1А, Одеса	0509940082	ОРЕНДА

Скріншот 7.1.5 — таблиця до видалення.



Назва фірми	ПІБ	Адреса	Телефон	Тип послуги
Vavion	Шевченко Микола Миколайович	Каманіна 12, Одеса	0509948088	ПРОДАЖ
Альянс	Бойко Олександр Валерійович	Одеської громади 2б, Одеса	0999825671	ПРОДАЖ
Золотий клюс	Горянська Аліна Андріївна	Маячний 7, Одеса	0995996152	ПРОДАЖ
Active	Ватолін Сергій Сергійович	Перемоги 17, Одеса	0508821151	ОРЕНДА
Vector	Іванча Марія Олександрівна	Нова 3, Одеса	0998526282	ОРЕНДА
Dream House	Дімітрєва Анна Сергіївна	Фонтанська дорога 22, Одеса	0502180062	ОРЕНДА
Realty Sale	Іщенко Анатолій Анатолійович	Ярослава Мудрого 2, Одеса	0509726170	ПРОДАЖ
Елемент	Мельник Денис Валерійович	Довга 1А, Одеса	0509940082	ОРЕНДА

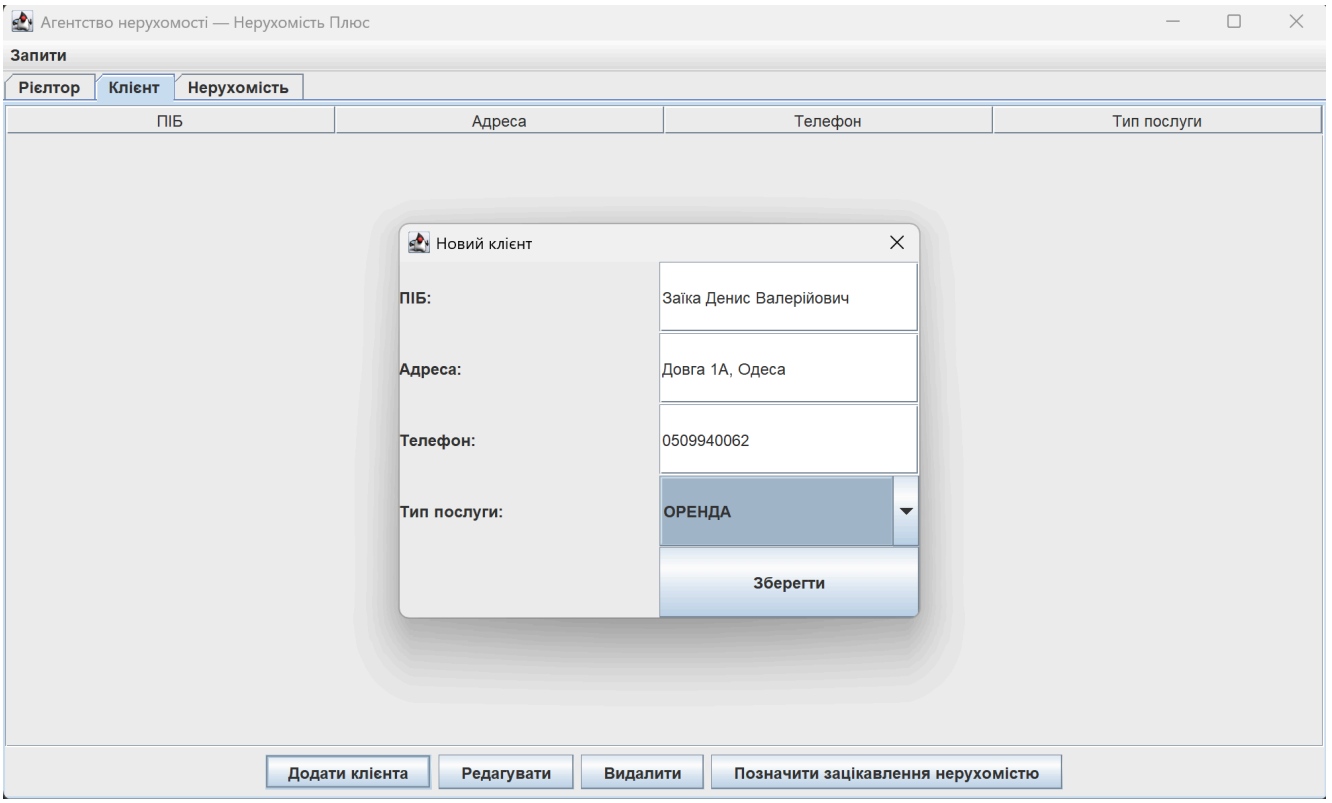
Скріншот 7.1.6 — таблиця після видалення

7.2. CRUD-операції для клієнта

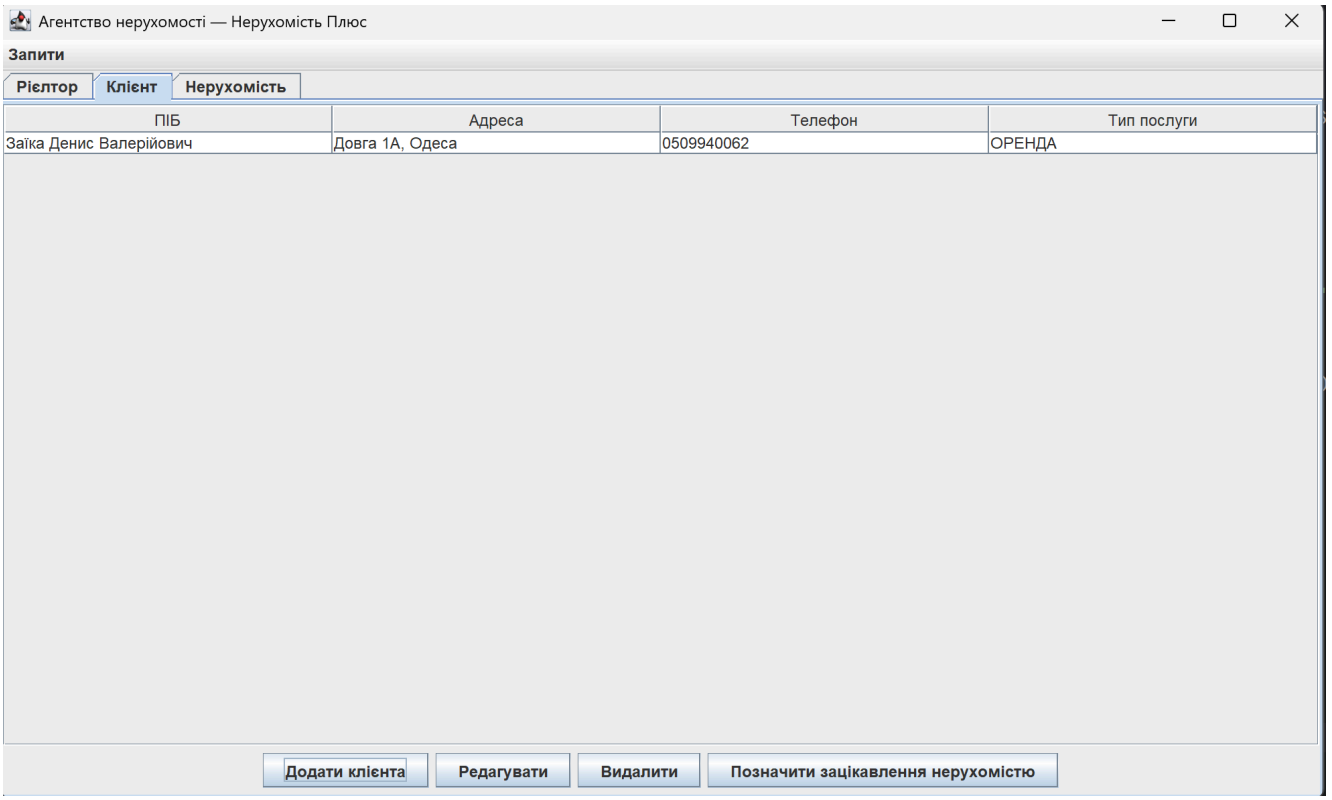
У цьому підпункті демонструється робота з вкладкою «Клієнт». Для перевірки функціоналу виконуються всі базові операції CRUD (Create, Read, Update, Delete). Додатково показано можливість позначення зацікавлення клієнтом у нерухомості.

1. Додавання клієнта (Create)

- **Опис:** при натисканні кнопки «Додати клієнта» відкривається діалогове вікно (JDialog) з полями для введення даних: ПІБ, адреса, телефон, тип послуги.
- **Дії користувача:** заповнити всі поля та натиснути «Зберегти».
- **Результат:** новий клієнт додається до списку агентства (Agency) та відображається у таблиці.



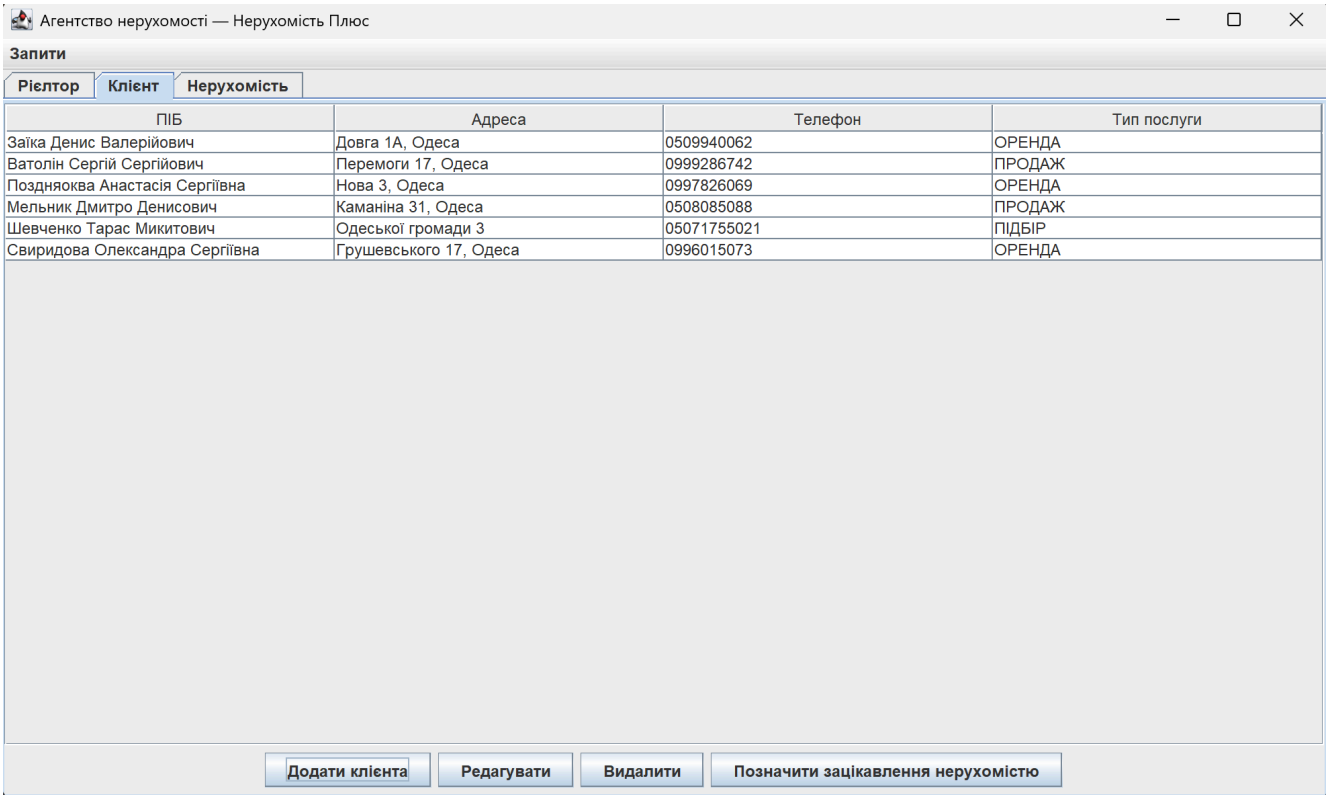
Скріншот 7.2.1 — форма додавання нового клієнта



Скріншот 7.2.2 — таблиця після додавання клієнта

2. Перегляд списку клієнтів (Read)

- **Опис:** таблиця у вкладці «Клієнт» відображає всі додані записи.
- **Дії користувача:** після додавання кількох клієнтів можна переглянути повний список.
- **Результат:** користувач бачить усіх клієнтів агентства у вигляді рядків таблиці.

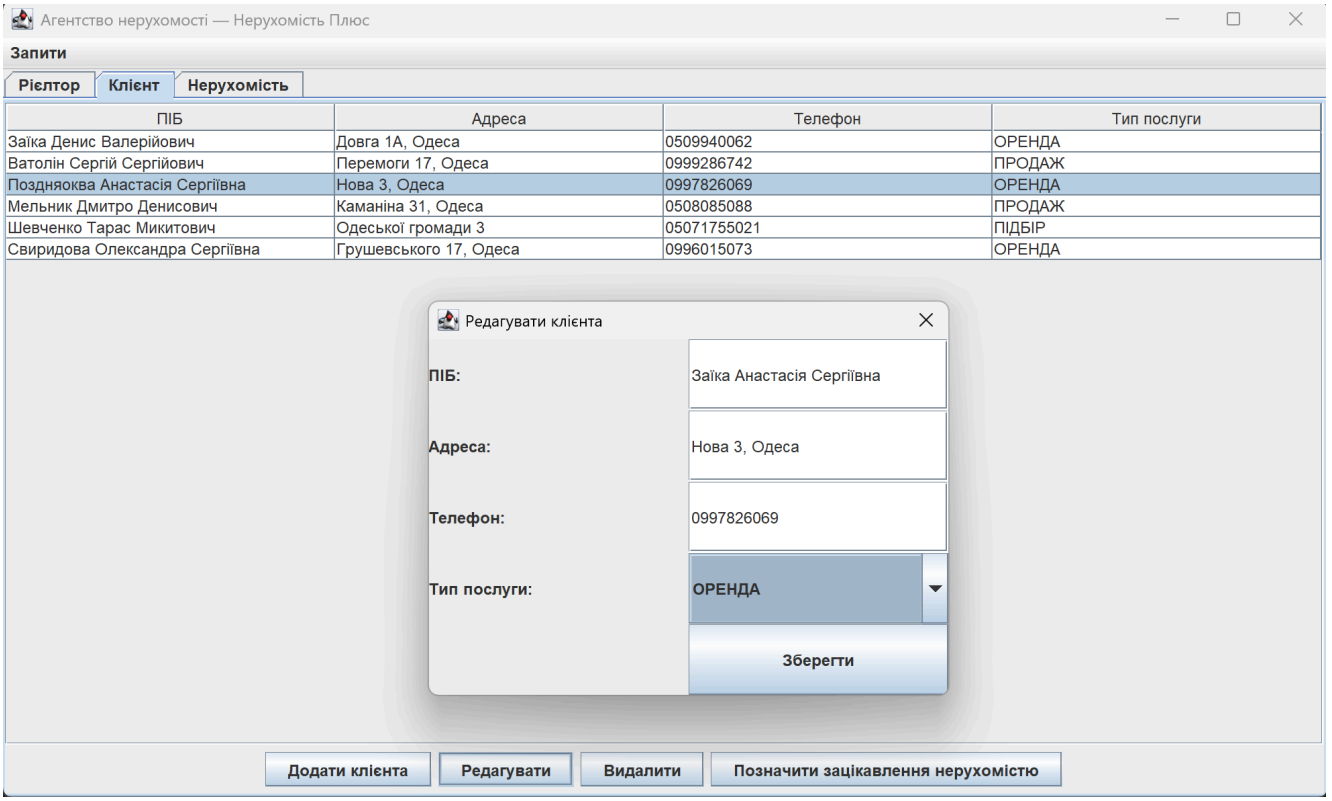


ПІБ	Адреса	Телефон	Тип послуги
Заїка Денис Валерійович	Довга 1А, Одеса	0509940062	ОРЕНДА
Ватолін Сергій Сергійович	Перемоги 17, Одеса	0999286742	ПРОДАЖ
Поздняоква Анастасія Сергіївна	Нова 3, Одеса	0997826069	ОРЕНДА
Мельник Дмитро Денисович	Каманіна 31, Одеса	0508085088	ПРОДАЖ
Шевченко Тарас Микитович	Одеської громади 3	05071755021	ПІДБІР
Свиридова Олександра Сергіївна	Грушевського 17, Одеса	0996015073	ОРЕНДА

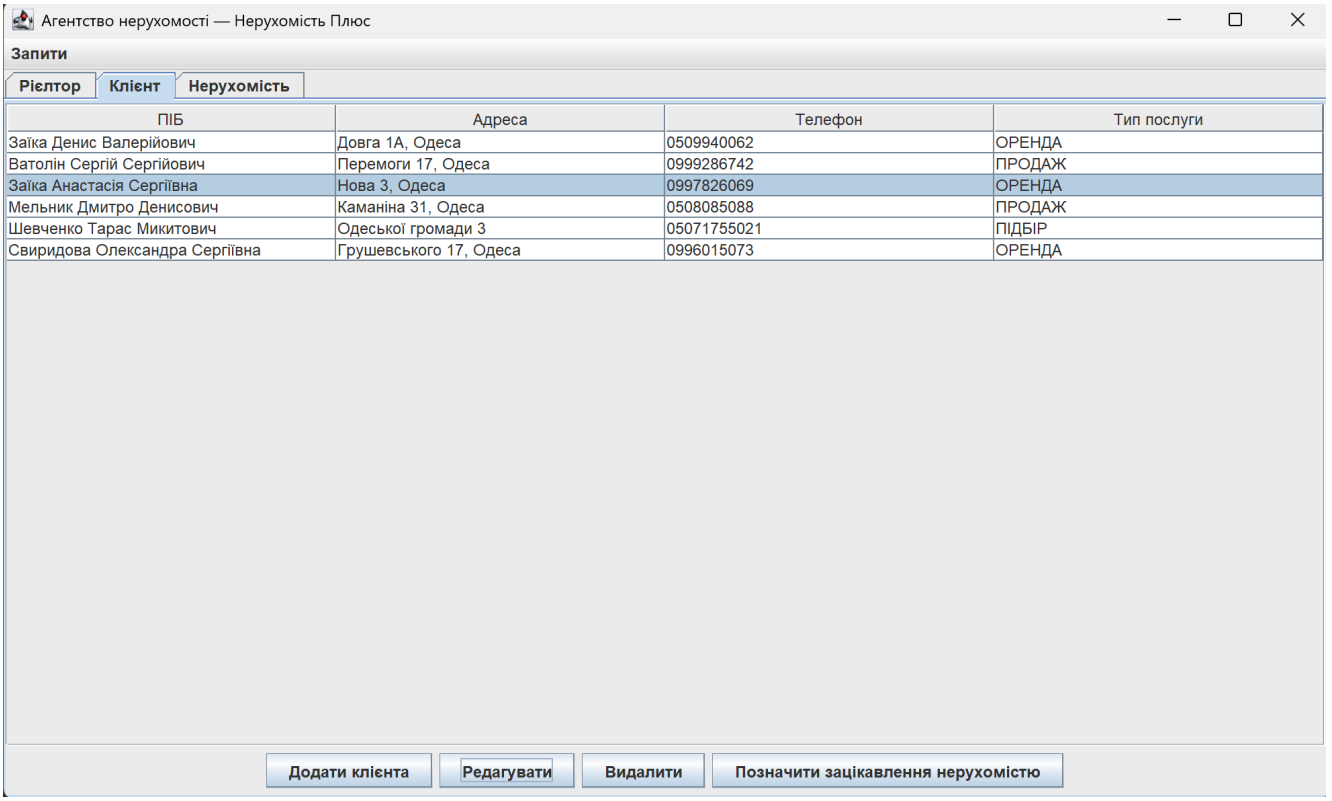
Скріншот 7.2.3 — таблиця зі списком кількох клієнтів

3. Редагування клієнта (Update)

- **Опис:** при натисканні кнопки «Редагувати» відкривається форма з уже заповненими даними обраного клієнта.
- **Дії користувача:** змінити потрібні поля (наприклад, адресу чи тип послуги) та натиснути «Зберегти».
- **Результат:** дані у таблиці оновлюються відповідно до змін.



Скріншот 7.2.4 — форма редагування клієнта

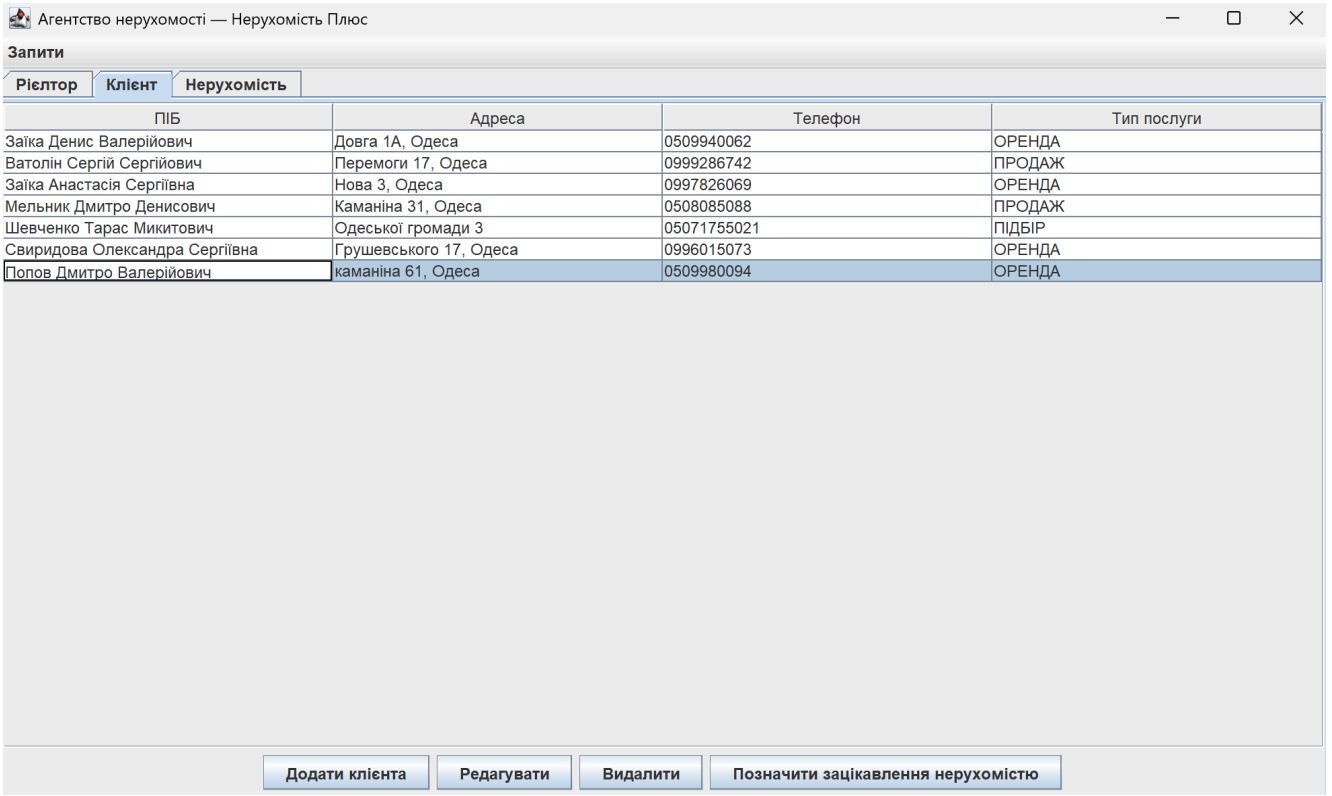


Скріншот 7.2.5 — таблиця після редагування

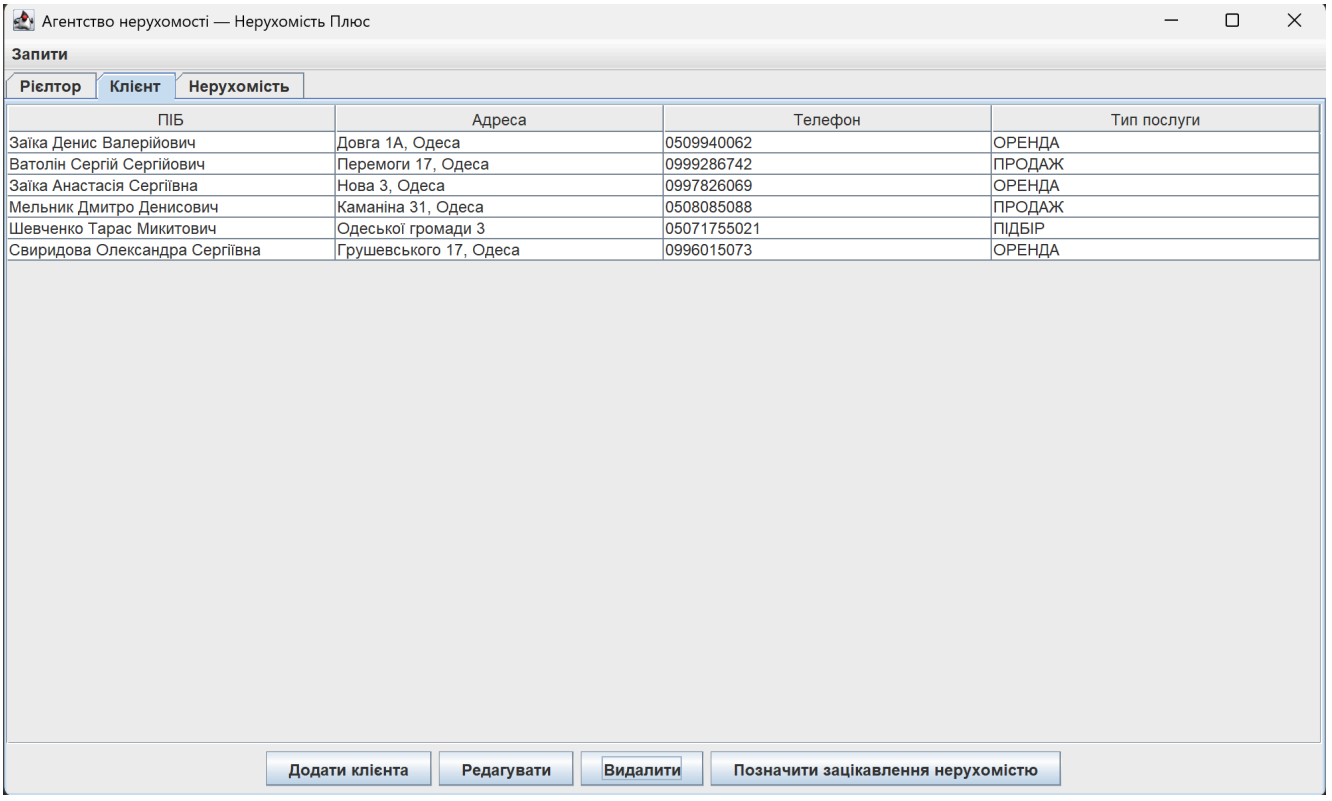
4. Видалення клієнта (Delete)

- **Опис:** при натисканні кнопки «Видалити» обраний клієнт видаляється зі списку агентства та з таблиці.

- **Дії користувача:** вибрати рядок у таблиці та натиснути «Видалити».
- **Результат:** відповідний рядок зникає з таблиці, а об’єкт видаляється зі списку у класі **Agency**.



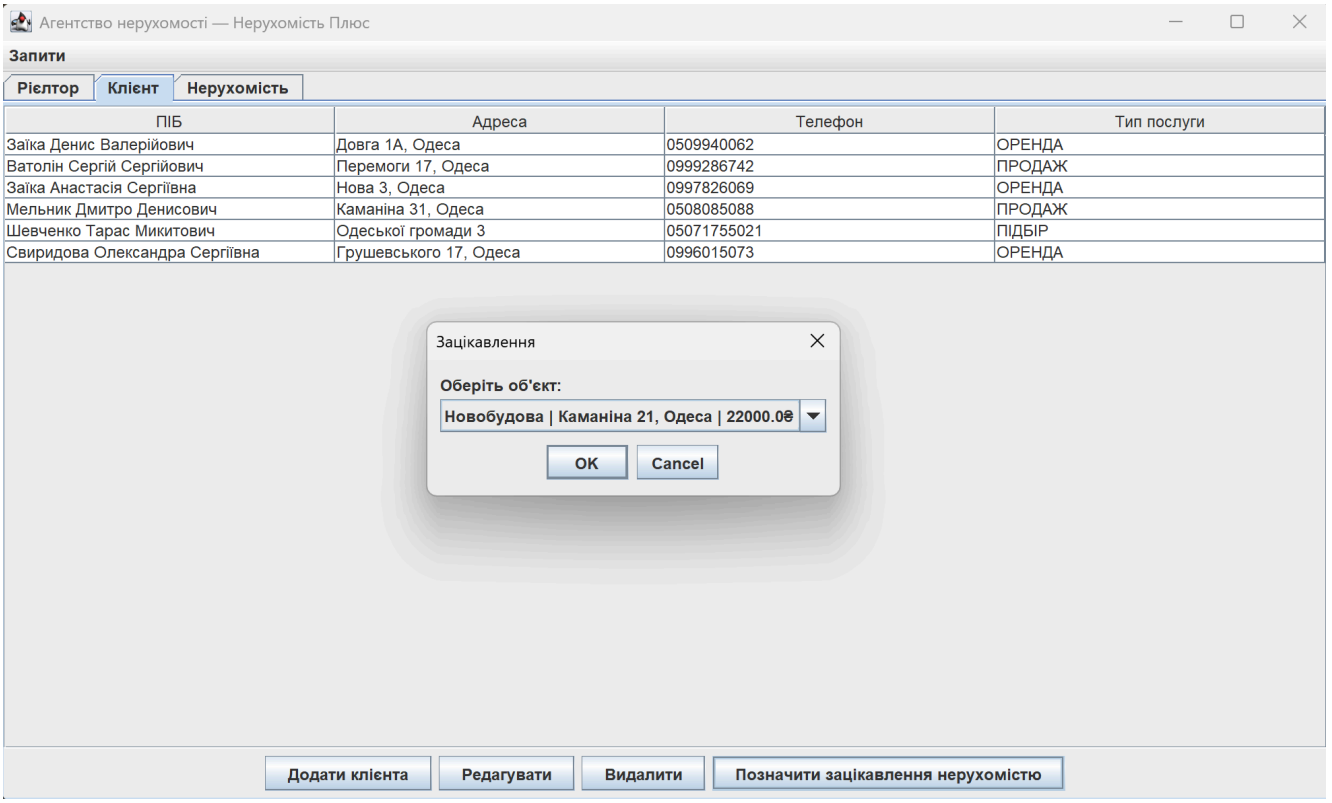
Скріншот 7.2.6 — таблиця до видалення



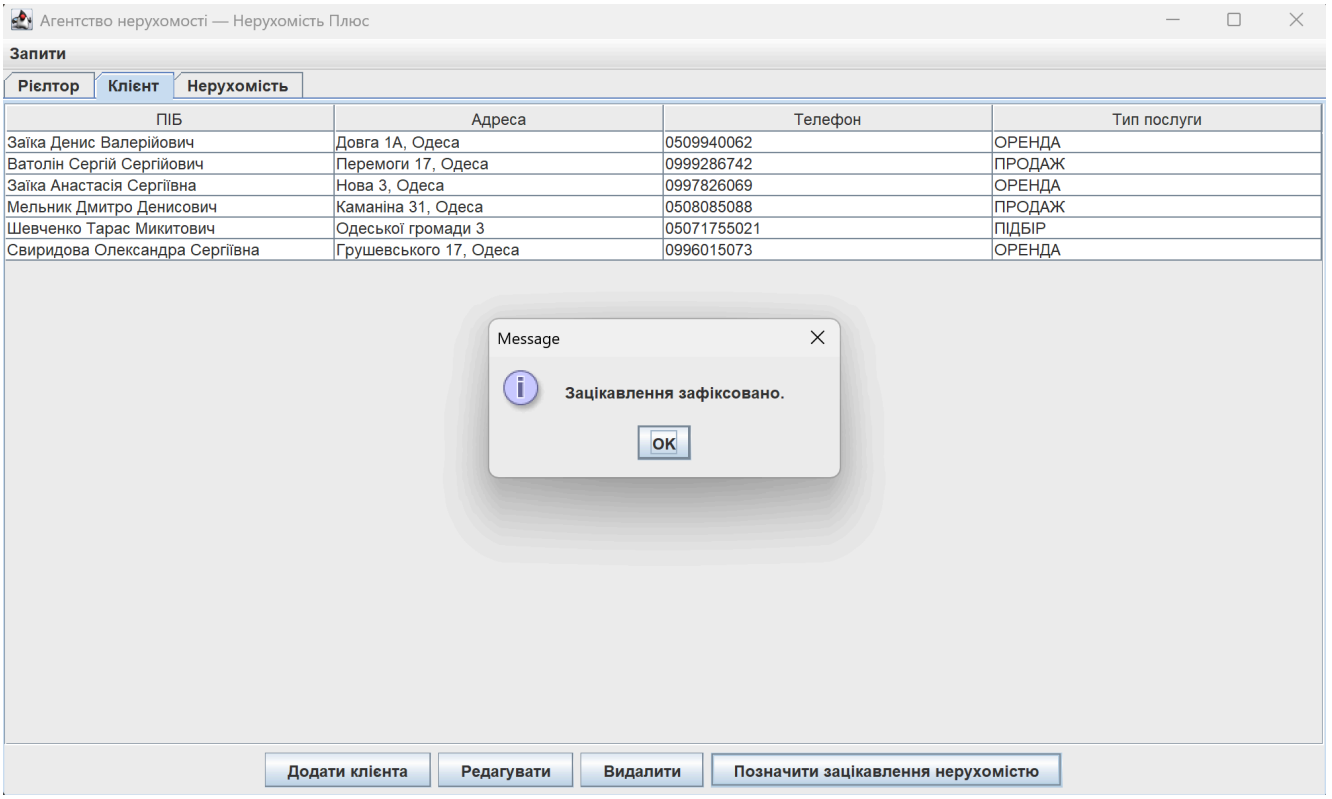
Скріншот 7.2.7 — таблиця після видалення

5. Позначення зацікавлення нерухомістю

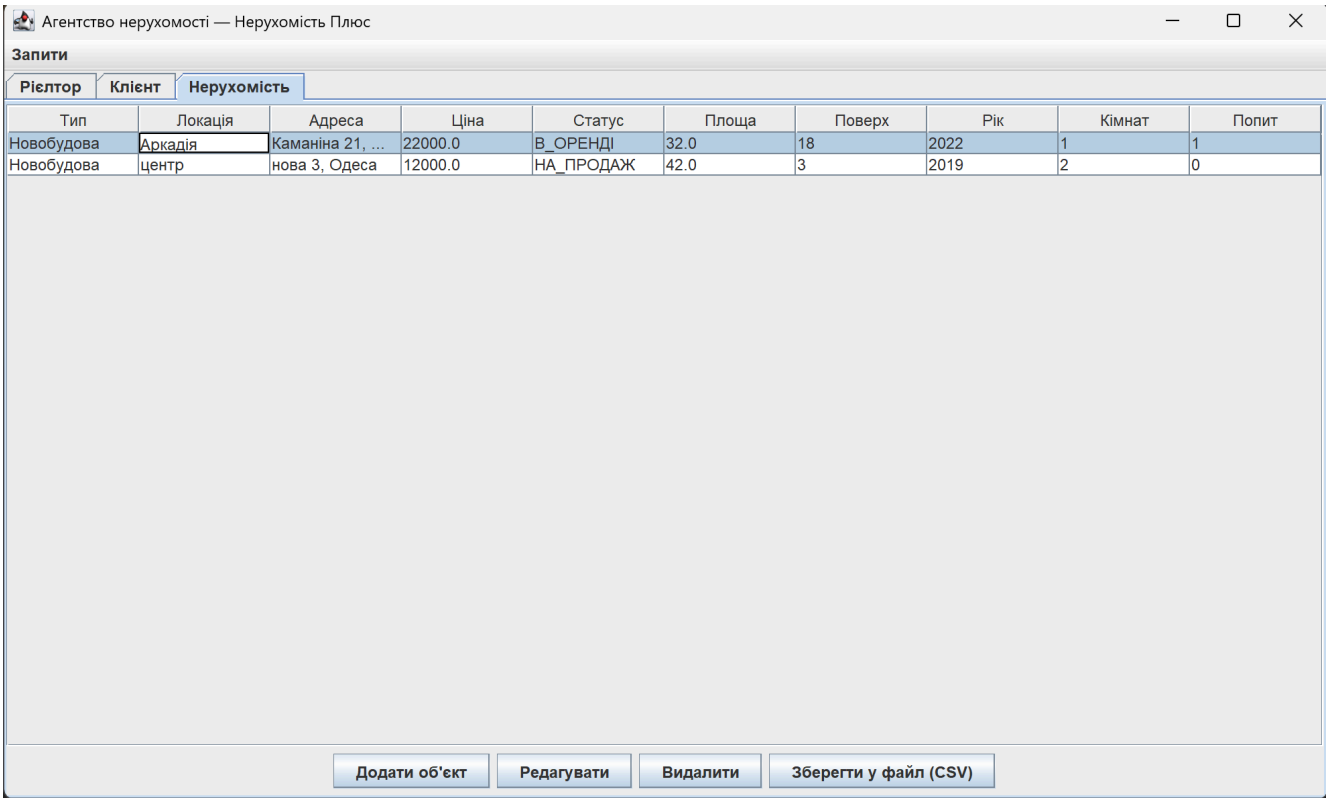
- **Опис:** у вкладці «Клієнт» є кнопка «Позначити зацікавлення нерухомістю». Вона відкриває діалог, де клієнт може вибрати об’єкт нерухомості.
- **Дії користувача:** вибрати потрібний об’єкт зі списку.
- **Результат:** у відповідному об’єкті класу **Property** збільшується поле **interestCount**, що відображає попит.



Скріншот 7.2.8 — діалог вибору об'єкта нерухомості для позначення зацікавлення.



Скріншот 7.2.9 — повідомлення про успішне позначення зацікавлення.



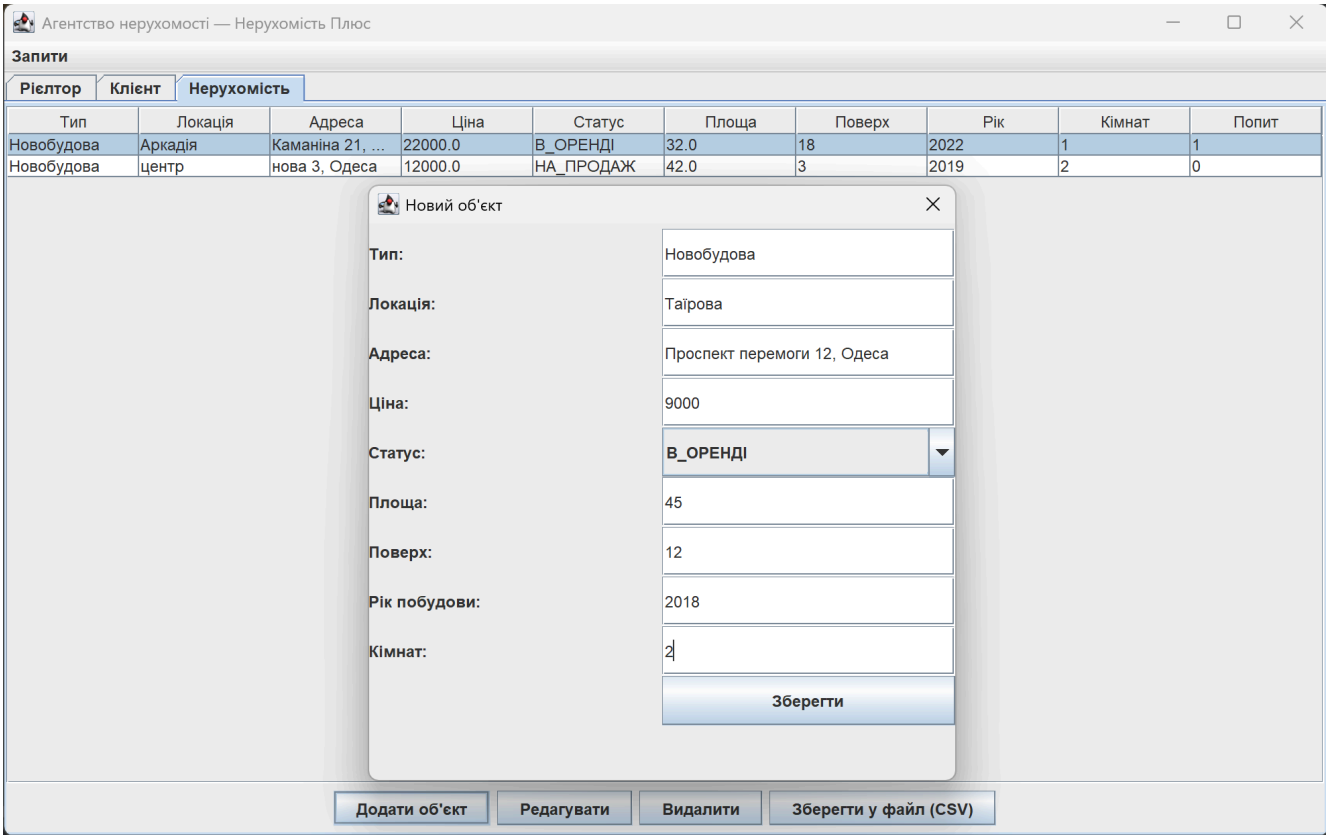
Скріншот 7.2.10 — таблиця нерухомості після позначення

7.3. CRUD-операції для нерухомості

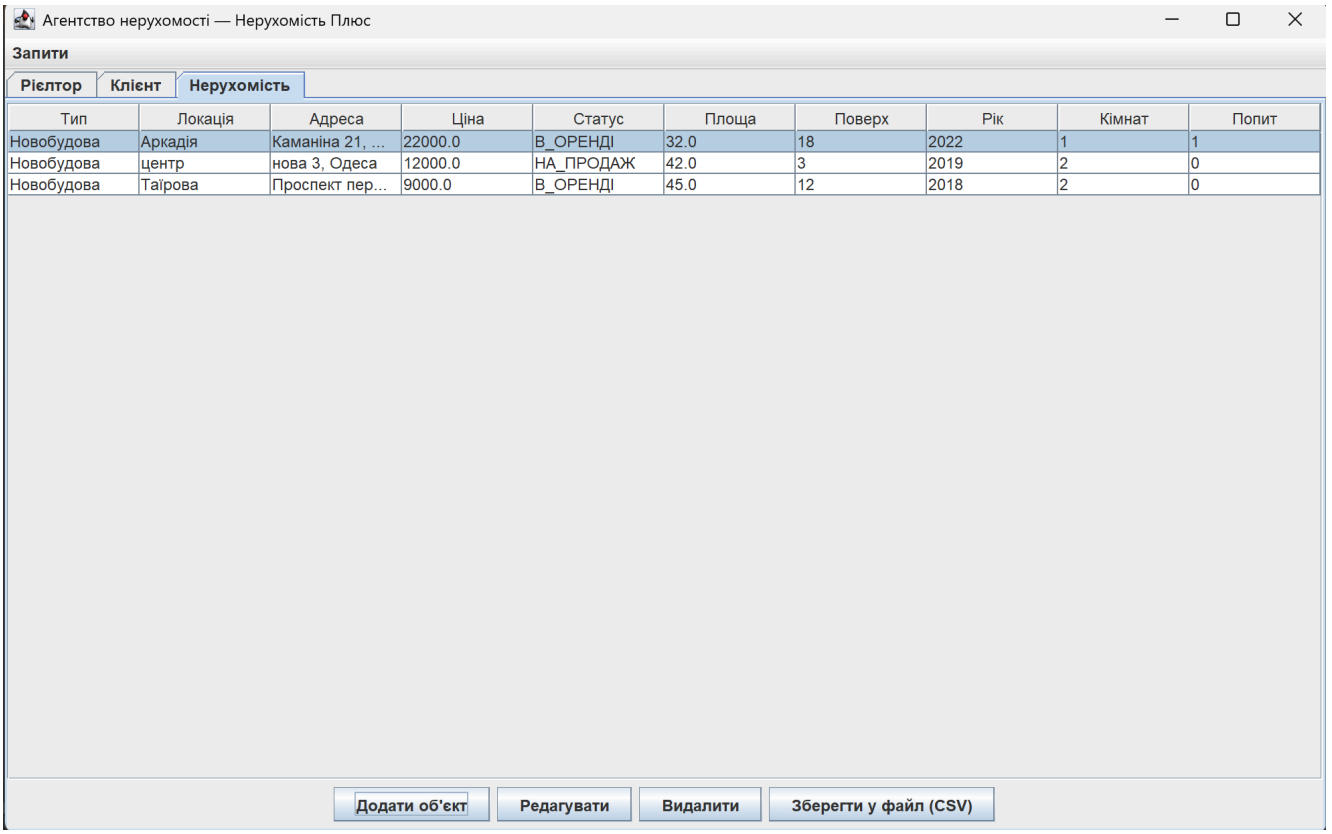
У цьому підпункті демонструється робота з вкладкою «Нерухомість». Для перевірки функціоналу виконуються всі базові операції CRUD (Create, Read, Update, Delete). Додатково показано можливість збереження даних у файл CSV.

1. Додавання об’єкта нерухомості (Create)

- **Опис:** при натисканні кнопки «Додати об’єкт» відкривається діалогове вікно (JDialog) з полями для введення даних: тип, локація, адреса, ціна, статус, площа, поверх, рік побудови, кількість кімнат.
- **Дії користувача**: заповнити всі поля та натиснути «Зберегти».
- **Результат:** новий об’єкт додається до списку агентства (Agency) та відображається у таблиці.



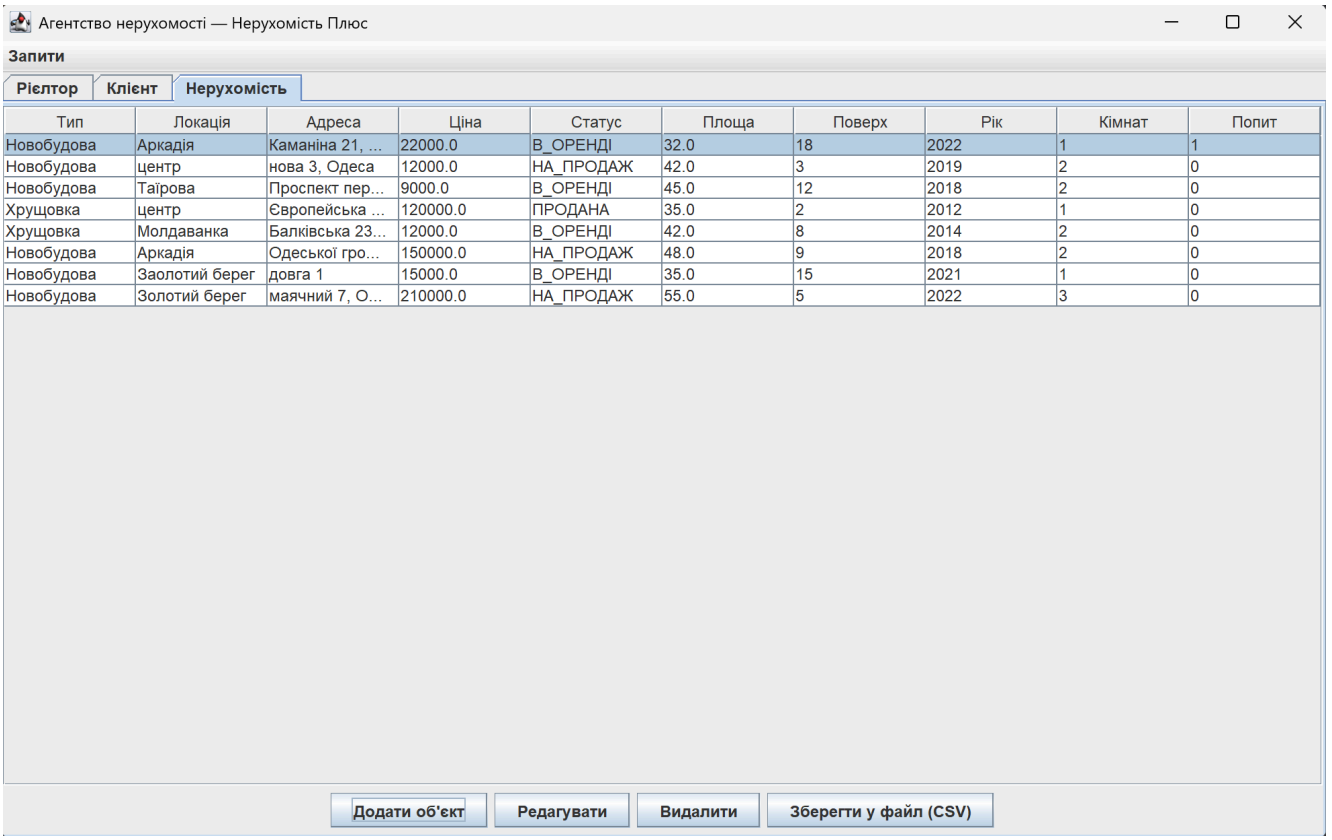
Скріншот 7.3.1 — форма додавання нового об’єкта нерухомості



Скріншот 7.3.2 — таблиця після додавання об’єкта

2. Перегляд списку об’єктів (Read)

- **Опис:** таблиця у вкладці «Нерухомість» відображає всі додані записи.
- **Дії користувача:** після додавання кількох об’єктів можна переглянути повний список.
- **Результат:** користувач бачить усі об’єкти агентства у вигляді рядків таблиці.

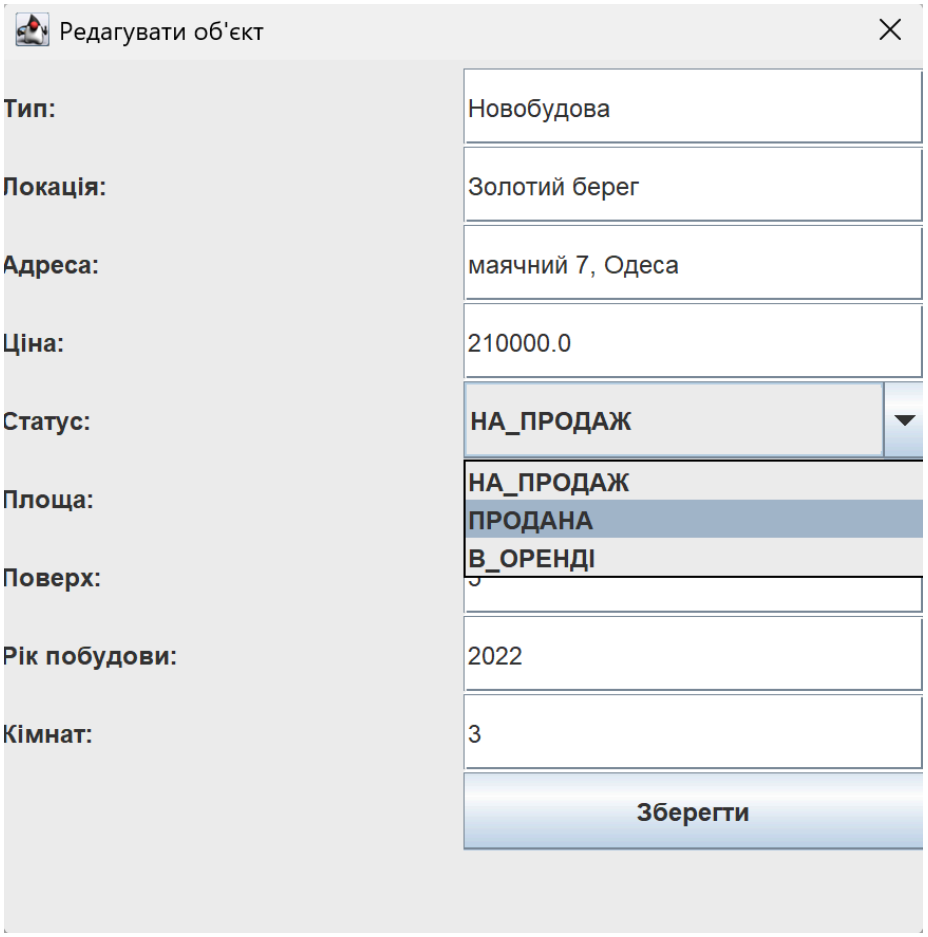


Тип	Локація	Адреса	Ціна	Статус	Площа	Поверх	Рік	Кімнат	Попит
Новобудова	Аркадія	Каманіна 21, ...	22000.0	В_ОРЕНДІ	32.0	18	2022	1	1
Новобудова	центр	нова 3, Одеса	12000.0	НА_ПРОДАЖ	42.0	3	2019	2	0
Новобудова	Таїрова	Проспект пер...	9000.0	В_ОРЕНДІ	45.0	12	2018	2	0
Хрущовка	центр	Європейська ...	120000.0	ПРОДАНА	35.0	2	2012	1	0
Хрущовка	Молдаванка	Балківська 23...	12000.0	В_ОРЕНДІ	42.0	8	2014	2	0
Новобудова	Аркадія	Одеської гро...	150000.0	НА_ПРОДАЖ	48.0	9	2018	2	0
Новобудова	Заолотий берег	довга 1	15000.0	В_ОРЕНДІ	35.0	15	2021	1	0
Новобудова	Золотий берег	маячний 7, О...	210000.0	НА_ПРОДАЖ	55.0	5	2022	3	0

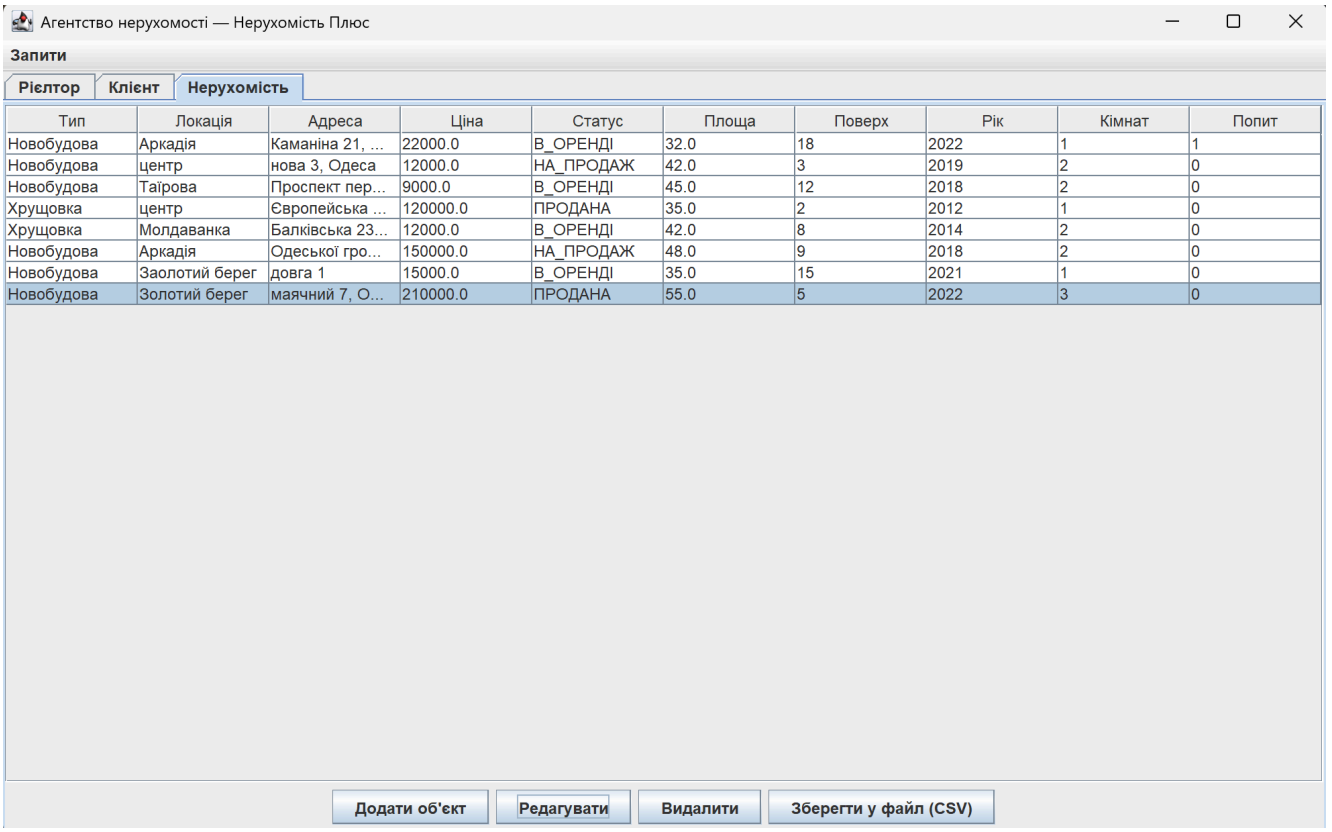
Скріншот 7.3.3 — таблиця зі списком кількох об’єктів нерухомості

3. Редагування об’єкта (Update)

- **Опис:** при натисканні кнопки «Редагувати» відкривається форма з уже заповненими даними обраного об’єкта.
- **Дії користувача:** змінити потрібні поля (наприклад, ціну чи статус) та натиснути «Зберегти».
- **Результат:** дані у таблиці оновлюються відповідно до змін.



Скріншот 7.3.4 — форма редагування об’єкта нерухомості

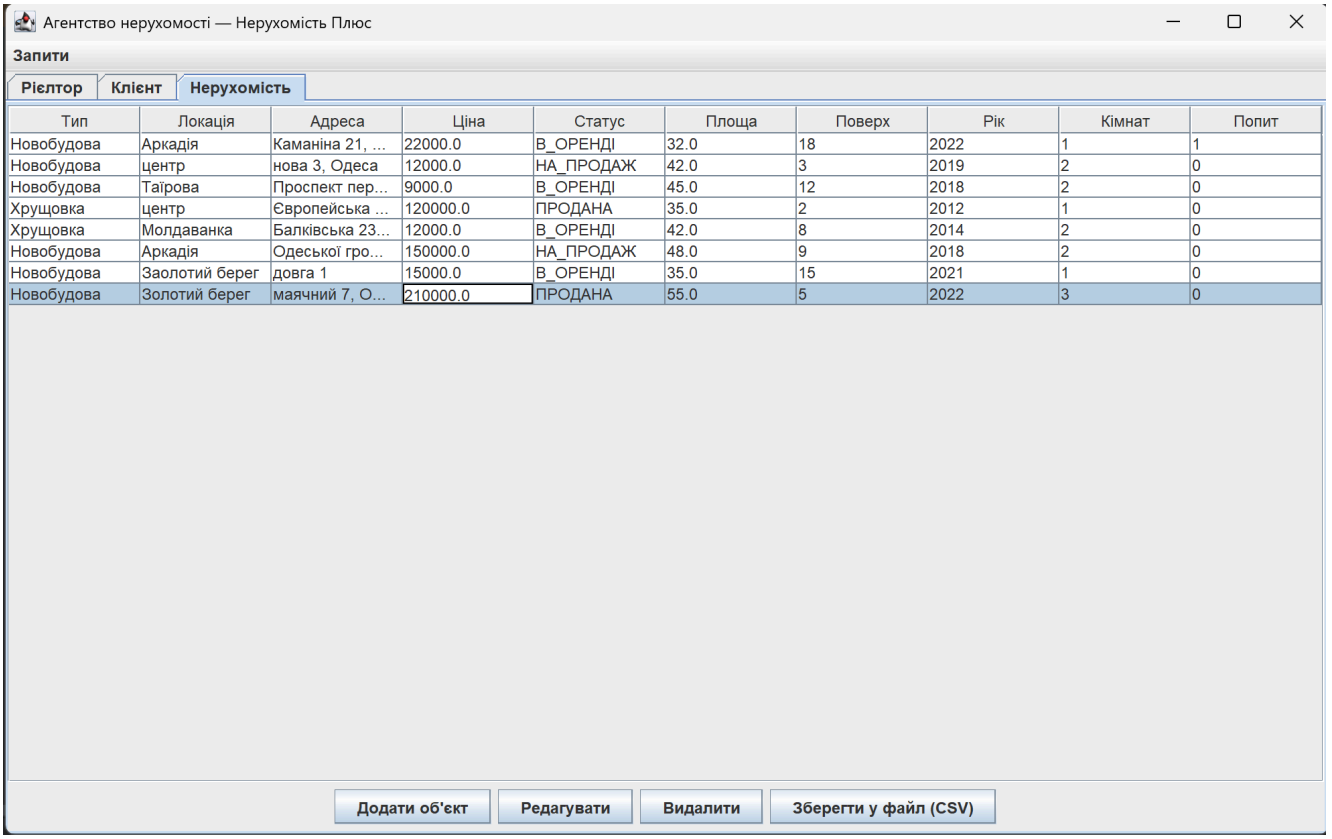


Тип	Локація	Адреса	Ціна	Статус	Площа	Поверх	Рік	Кімнат	Попит
Новобудова	Аркадія	Каманіна 21, ...	22000.0	В_ОРЕНДІ	32.0	18	2022	1	1
Новобудова	центр	нова 3, Одеса	12000.0	НА_ПРОДАЖ	42.0	3	2019	2	0
Новобудова	Таїрова	Проспект пер...	9000.0	В_ОРЕНДІ	45.0	12	2018	2	0
Хрущовка	центр	Європейська ...	120000.0	ПРОДАНА	35.0	2	2012	1	0
Хрущовка	Молдаванка	Балківська 23...	12000.0	В_ОРЕНДІ	42.0	8	2014	2	0
Новобудова	Аркадія	Одеської гро...	150000.0	НА_ПРОДАЖ	48.0	9	2018	2	0
Новобудова	Заолотий берег	довга 1	15000.0	В_ОРЕНДІ	35.0	15	2021	1	0
Новобудова	Золотий берег	маячний 7, О...	210000.0	ПРОДАНА	55.0	5	2022	3	0

Скріншот 7.3.5 — таблиця після редагування

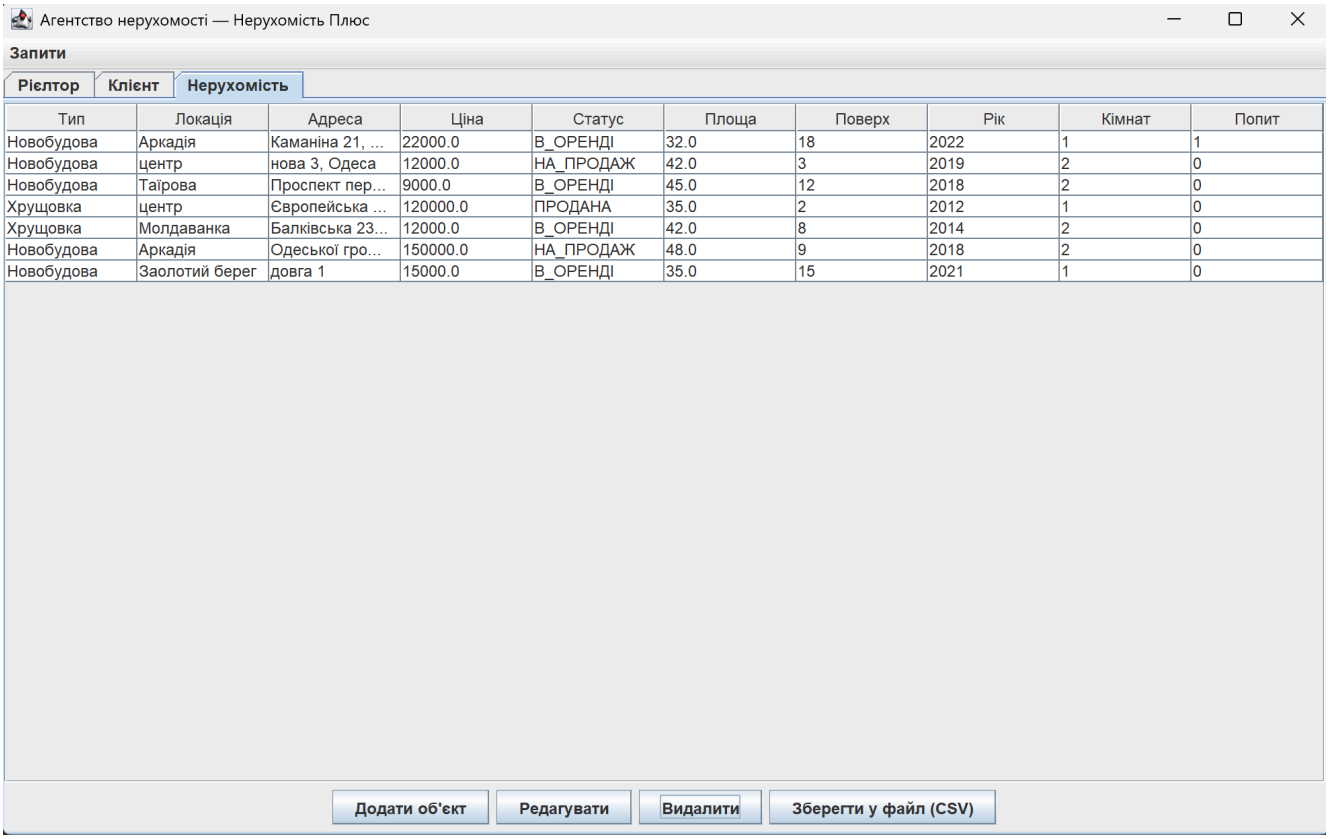
4. Видалення об’єкта (Delete)

- **Опис:** при натисканні кнопки «Видалити» обраний об’єкт видаляється зі списку агентства та з таблиці.
- **Дії користувача:** вибрати рядок у таблиці та натиснути «Видалити».
- **Результат:** відповідний рядок зникає з таблиці, а об’єкт видаляється зі списку у класі **Agency**.



Тип	Локація	Адреса	Ціна	Статус	Площа	Поверх	Рік	Кімнат	Попит
Новобудова	Аркадія	Каманіна 21, ...	22000.0	В_ОРЕНДІ	32.0	18	2022	1	1
Новобудова	центр	нова 3, Одеса	12000.0	НА_ПРОДАЖ	42.0	3	2019	2	0
Новобудова	Таїрова	Проспект пер...	9000.0	В_ОРЕНДІ	45.0	12	2018	2	0
Хрущовка	центр	Європейська ...	120000.0	ПРОДАНА	35.0	2	2012	1	0
Хрущовка	Молдаванка	Балківська 23...	12000.0	В_ОРЕНДІ	42.0	8	2014	2	0
Новобудова	Аркадія	Одеської гро...	150000.0	НА_ПРОДАЖ	48.0	9	2018	2	0
Новобудова	Заолотий берег	довга 1	15000.0	В_ОРЕНДІ	35.0	15	2021	1	0
Новобудова	Золотий берег	маячний 7, О...	210000.0	ПРОДАНА	55.0	5	2022	3	0

Скріншот 7.3.6 — таблиця до видалення

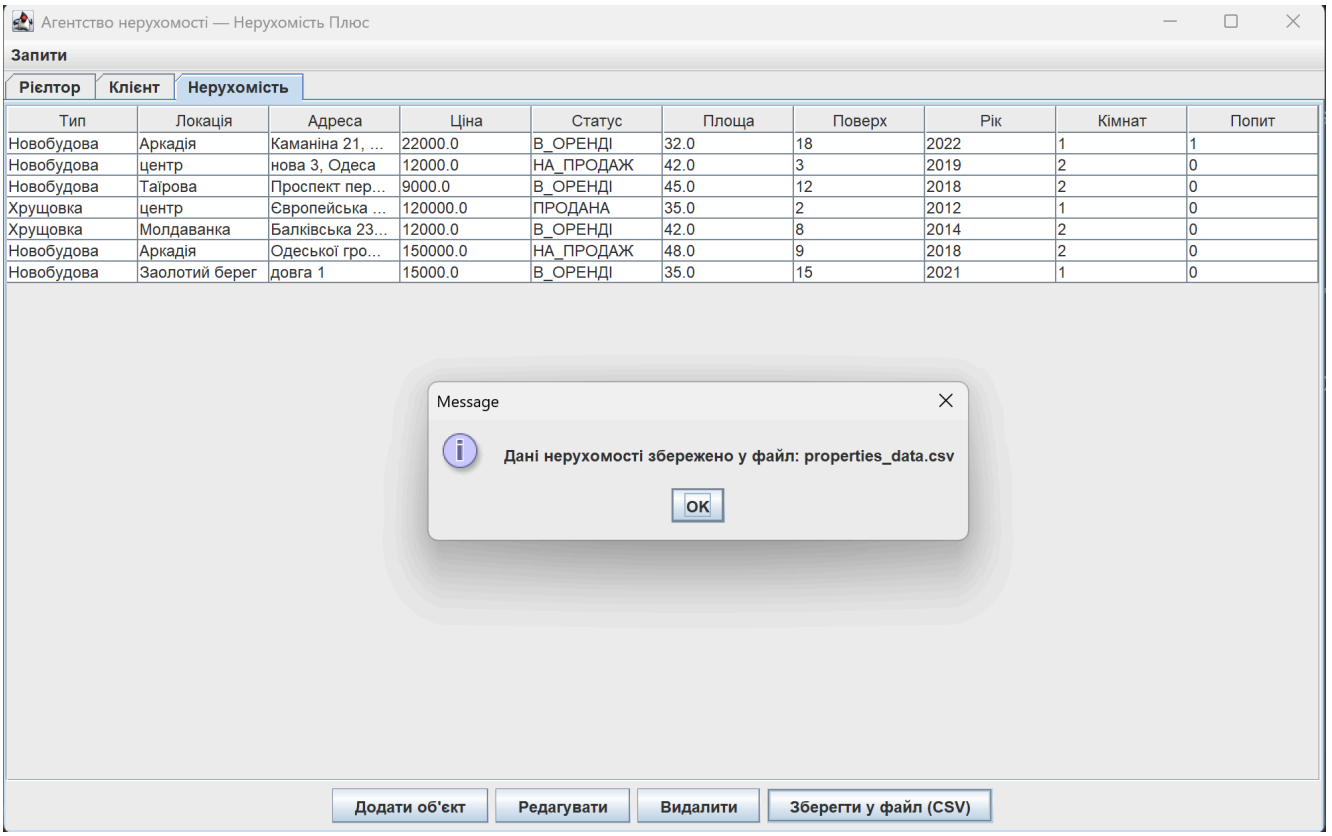


Тип	Локація	Адреса	Ціна	Статус	Площа	Поверх	Рік	Кімнат	Попит
Новобудова	Аркадія	Каманіна 21, ...	22000.0	В_ОРЕНДІ	32.0	18	2022	1	1
Новобудова	центр	нова 3, Одеса	12000.0	НА_ПРОДАЖ	42.0	3	2019	2	0
Новобудова	Таїрова	Проспект пер...	9000.0	В_ОРЕНДІ	45.0	12	2018	2	0
Хрущовка	центр	Європейська ...	120000.0	ПРОДАНА	35.0	2	2012	1	0
Хрущовка	Молдаванка	Балківська 23...	12000.0	В_ОРЕНДІ	42.0	8	2014	2	0
Новобудова	Аркадія	Одеської гро...	150000.0	НА_ПРОДАЖ	48.0	9	2018	2	0
Новобудова	Заолотий берег	довга 1	15000.0	В_ОРЕНДІ	35.0	15	2021	1	0

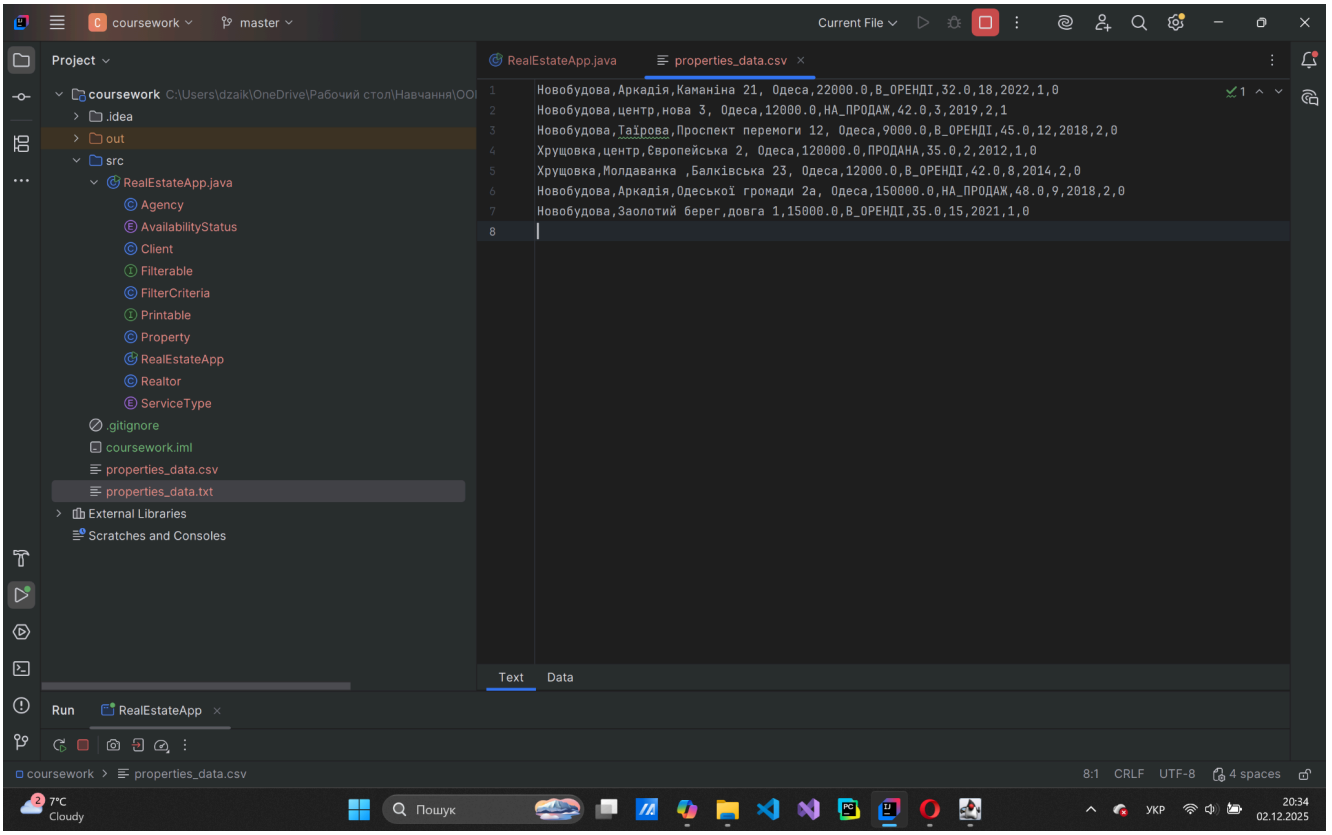
Скріншот 7.3.7 — таблиця після видалення

5. Збереження у файл (CSV)

- **Опис:** при натисканні кнопки «Зберегти у файл (CSV)» викликається метод **savePropertiesToFile()** класу **Agency**.
- **Дії користувача:** натиснути кнопку «Зберегти у файл».
- **Результат:** усі об’єкти нерухомості записуються у текстовий файл у форматі CSV.



Скріншот 7.3.8 — повідомлення про успішне збереження у файл.



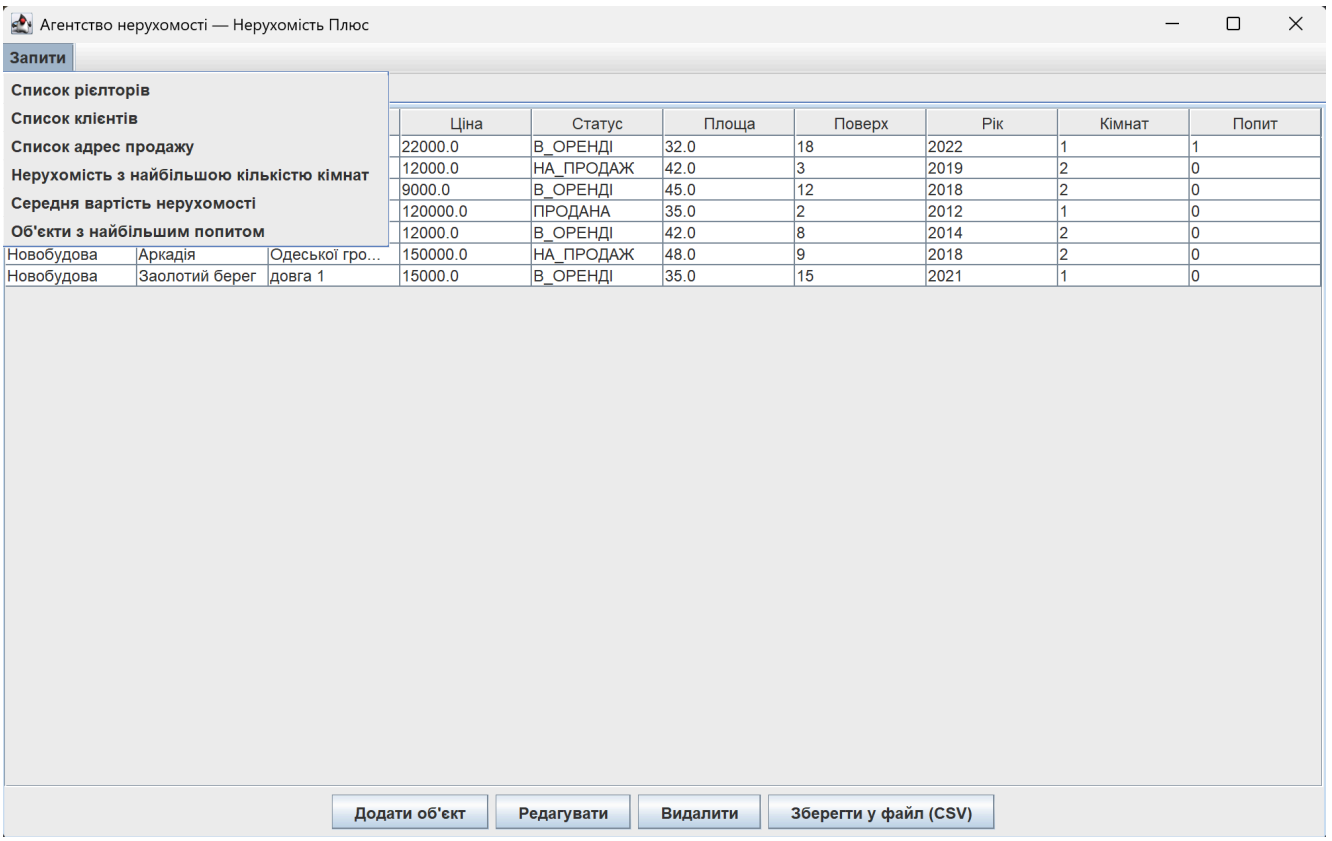
Скріншот 7.3.9 — вміст CSV-файлу з даними нерухомості.

7.4. Запити до системи (списки, середня ціна, попит тощо)

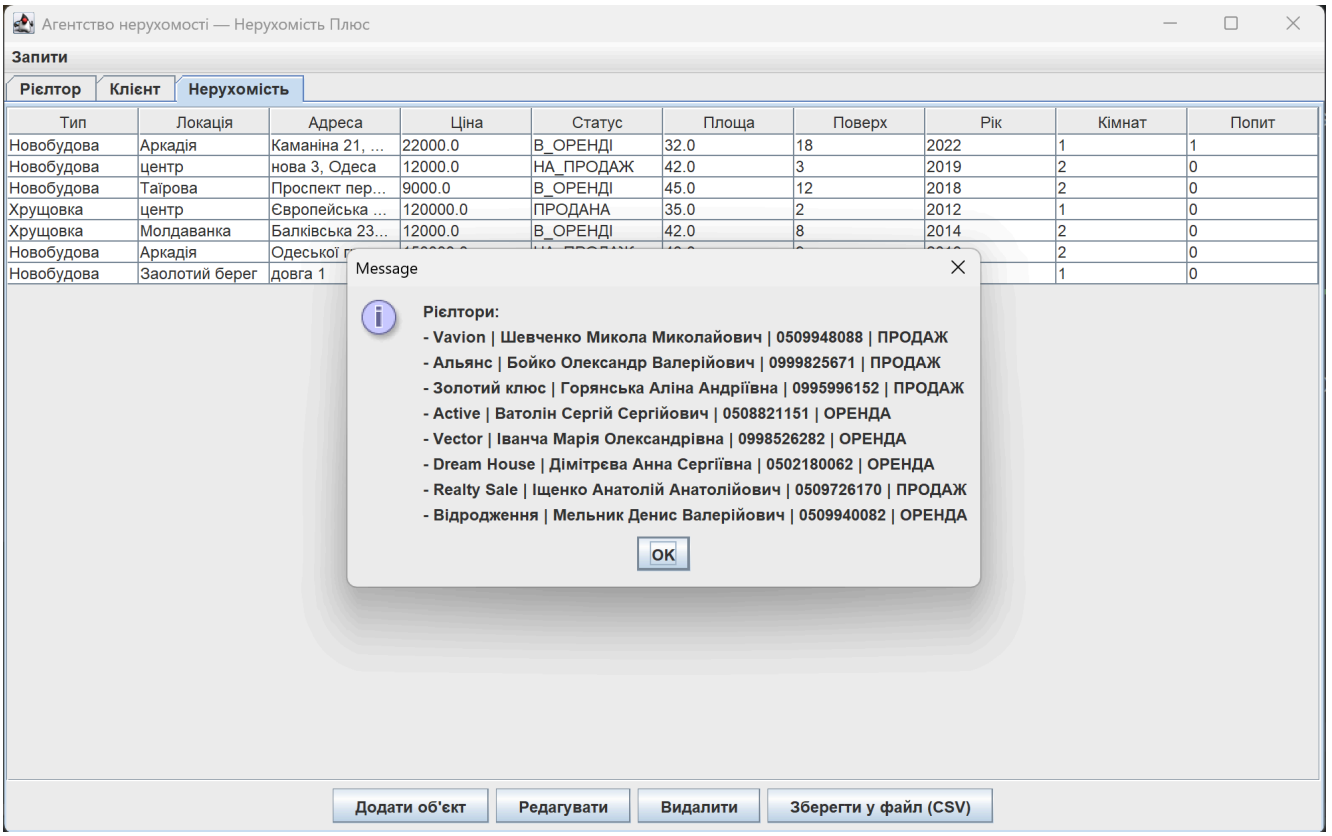
У цьому підпункті демонструється робота меню «Запити», яке реалізовано у головному класі RealEstateApp. Меню дозволяє користувачу швидко отримати статистику та списки, використовуючи методи класу Agency.

1. Список рієлторів

- Опис: при виборі пункту «Список рієлторів» у меню програма формує перелік усіх рієлторів агентства.
- Результат: користувач бачить список із назвою фірми, ПІБ, телефоном та типом послуги.



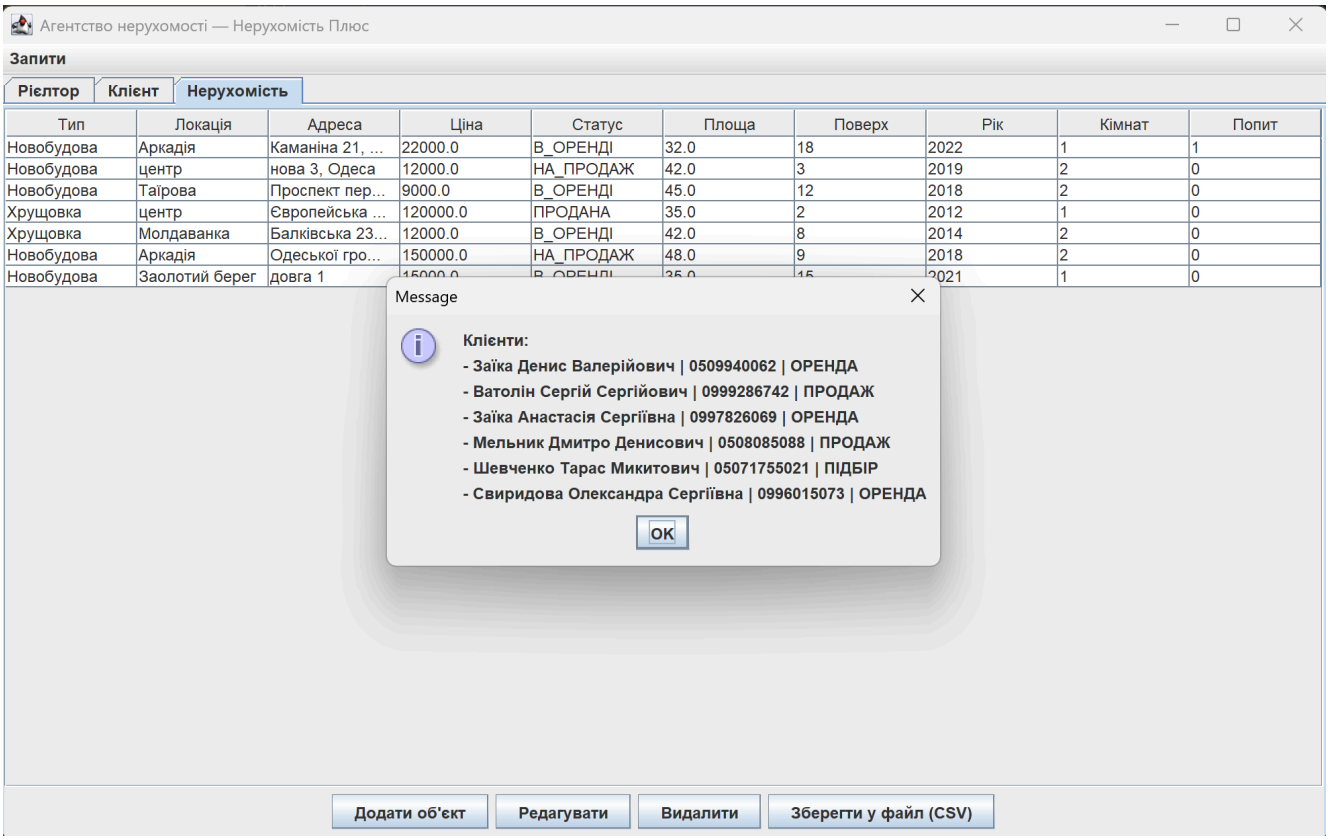
			Ціна	Статус	Площа	Поверх	Рік	Кімнат	Попит
			22000.0	В_ОРЕНДІ	32.0	18	2022	1	1
			12000.0	НА_ПРОДАЖ	42.0	3	2019	2	0
			9000.0	В_ОРЕНДІ	45.0	12	2018	2	0
			120000.0	ПРОДАНА	35.0	2	2012	1	0
			12000.0	В_ОРЕНДІ	42.0	8	2014	2	0
Новобудова	Аркадія	Одеської гро...	150000.0	НА_ПРОДАЖ	48.0	9	2018	2	0
Новобудова	Заолотий берег	довга 1	15000.0	В_ОРЕНДІ	35.0	15	2021	1	0



Скріншот 7.4.1 — вивід списку рієлторів у діалоговому вікні

2. Список клієнтів

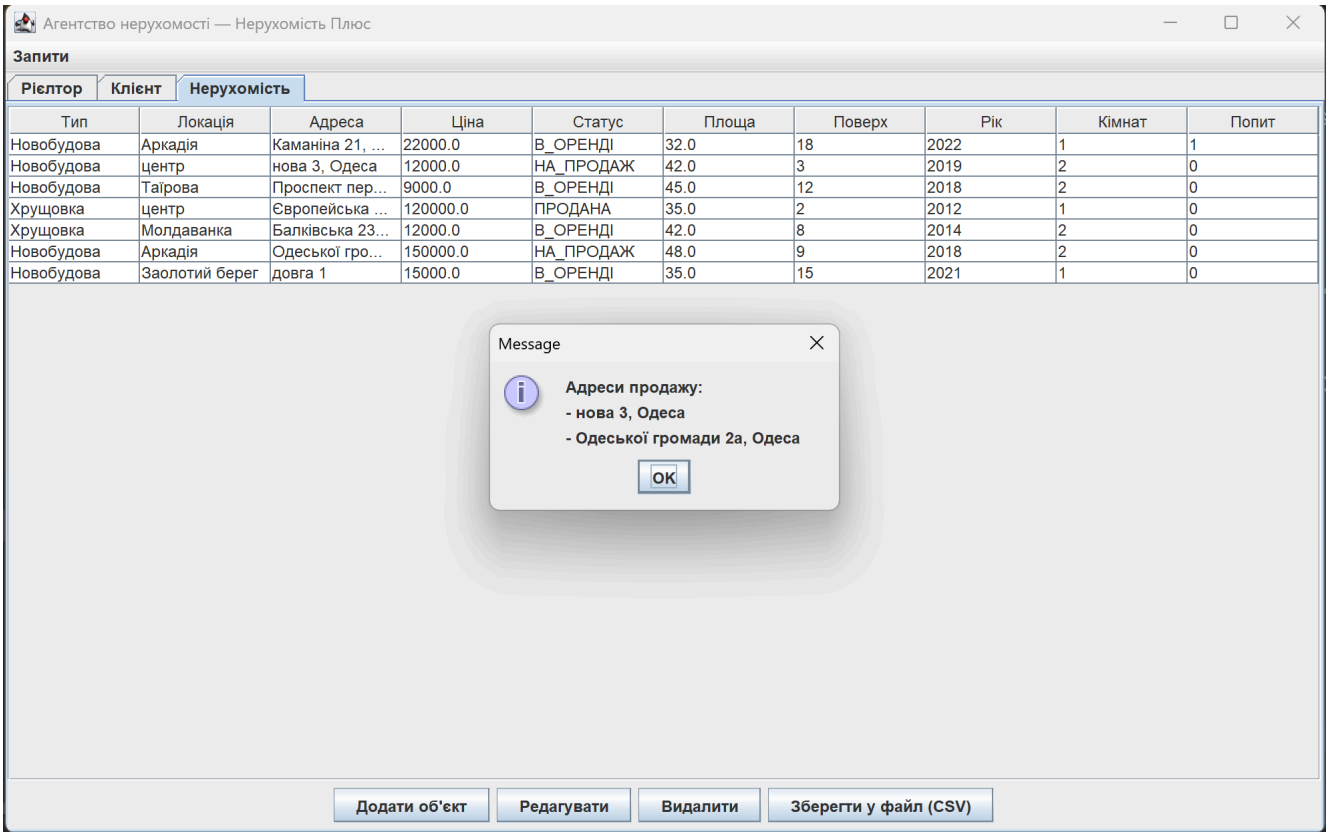
- Опис: при виборі пункту «Список клієнтів» у меню програма формує перелік усіх клієнтів агентства.
- Результат: користувач бачить список із ПІБ, адресою, телефоном та типом послуги.



Скріншот 7.4.2 — вивід списку клієнтів у діалоговому вікні

3. Список адрес продажу

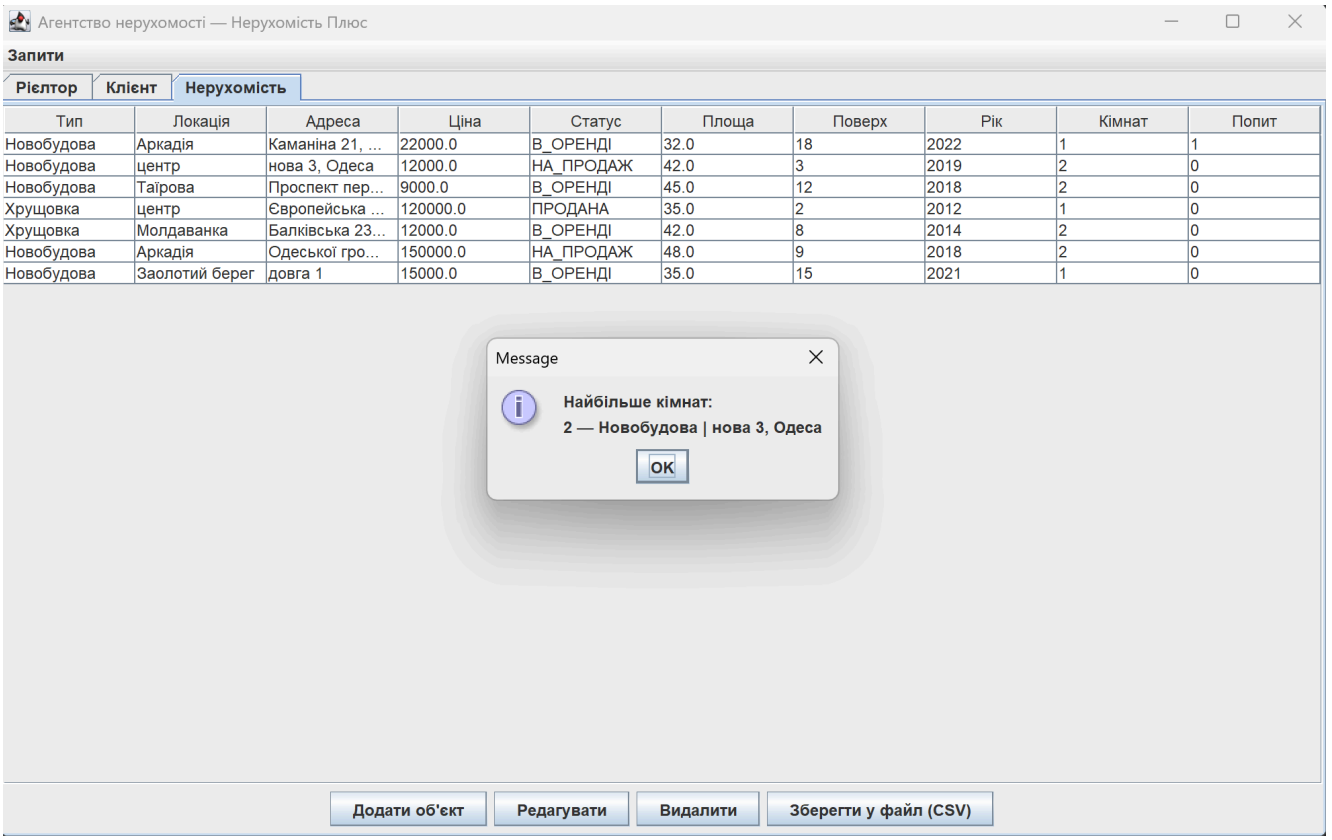
- **Опис:** при виборі пункту «Список адрес продажу» програма показує всі об’єкти нерухомості зі статусом НА_ПРОДАЖ.
- **Результат:** користувач отримує перелік адрес, які доступні для купівлі.



Скріншот 7.4.3 — список адрес нерухомості зі статусом «НА_ПРОДАЖ»

4. Нерухомість з найбільшою кількістю кімнат

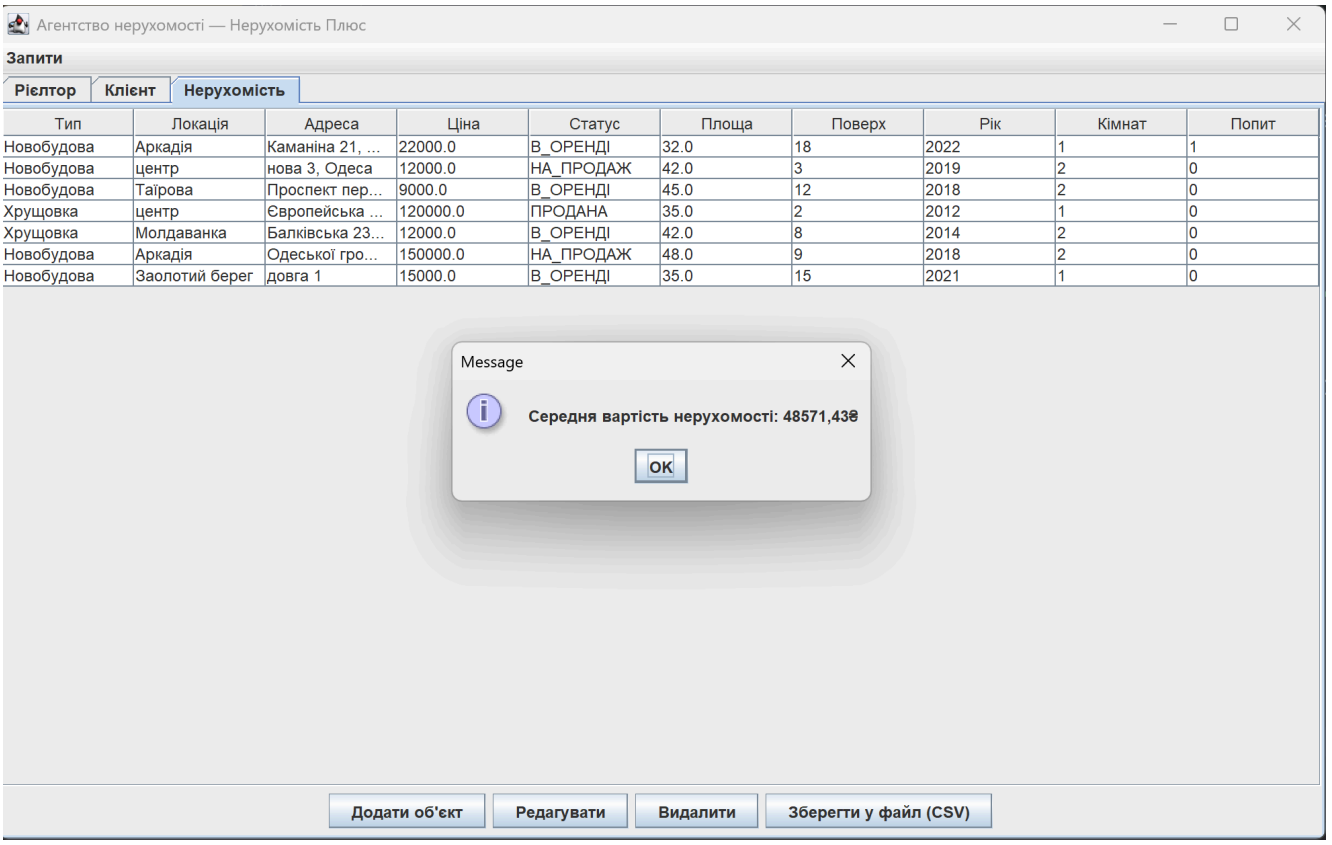
- **Опис:** при виборі пункту «Нерухомість з найбільшою кількістю кімнат» викликається метод propertyWithMostRooms().
- **Результат:** користувач бачить інформацію про об’єкт із найбільшою кількістю кімнат.



Скріншот 7.4.4 — повідомлення з даними про об’єкт із найбільшою кількістю кімнат.

5. Середня вартість нерухомості

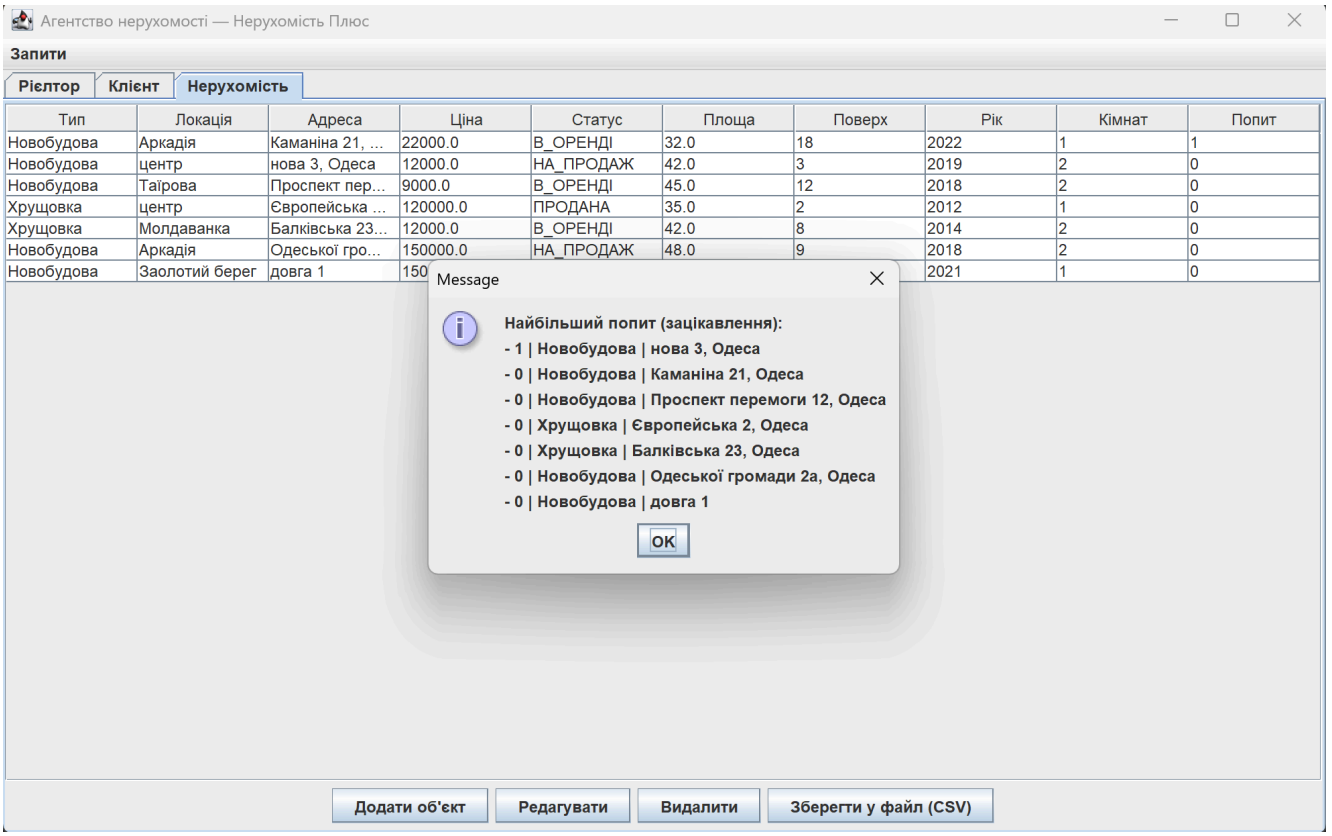
- **Опис:** при виборі пункту «Середня вартість нерухомості» викликається метод averagePrice().
- **Результат:** користувач отримує повідомлення із середньою ціною всіх об’єктів у системі.



Скріншот 7.4.5 — повідомлення з розрахованою середньою ціною

6. Об’єкти з найбільшим попитом

- **Опис:** при виборі пункту «Об’єкти з найбільшим попитом» викликається метод mostDemandedProperties().
- **Результат:** користувач бачить список об’єктів, відсортованих за рівнем попиту (interestCount).



Скріншот 7.4.6 — список об’єктів нерухомості з найбільшим попитом

8. Висновки

8.1. Досягнуті результати

У результаті курсової роботи створено повноцінну програму «Агентство нерухомості» з графічним інтерфейсом на базі Swing, чіткою об'єктною моделлю та набором інструментів для управління даними агентства. Програма охоплює три сутності (Рієлтор, Клієнт, Нерухомість), підтримує CRUD-операції, збереження у CSV, статистичні запити та базову взаємодію користувача через діалогові вікна.

Архітектура та модель даних

- **Структура класів:** реалізовано п'ять ключових компонентів: Property, Client, Realtor, Agency (менеджер даних) та RealEstateApp (GUI).
- **Перерахування:** додано enum-и ServiceType (тип послуги) та AvailabilityStatus (статус доступності нерухомості), що підвищує типобезпеку та узгодженість даних.
- **Інтерфейси розширення:** Printable для консольного виводу та Filterable для логічної фільтрації Property за критеріями (тип, локація, діапазон ціни, статус).
- **Поле попиту:** у Property запроваджено interestCount з методом markInterest(), що дозволяє відстежувати зацікавлення клієнтів.

Графічний інтерфейс (Swing) і взаємодія

- **Три вкладки (JTabbedPane):** «Рієлтор», «Клієнт», «Нерухомість» — кожна має JTable з відповідною DefaultTableModel та панель керування.
- **Діалоги (JDialog):** для додавання/редагування сутностей з валідацією числових полів у нерухомості; повідомлення про помилки через JOptionPane.
- **Меню «Запити» (JMenuBar):** швидкий доступ до списків і статистики (рієлтори, клієнти, адреси продажу, найбільше кімнат, середня ціна, найбільший попит).
- **Потокобезпечний запуск:** використано SwingUtilities.invokeLater(...) для коректного старту GUI.

CRUD-операції та збереження даних

- **Повний CRUD для трьох сутностей:** додавання, перегляд, редагування, видалення з синхронізацією таблиць і внутрішніх списків у Agency.
- **Експорт у CSV:** метод Agency.savePropertiesToFile(...) зберігає об’єкти нерухомості з усіма полями, включно з «Попит».
- **Позначення зацікавлення:** окремий користувацький потік дій у вкладці «Клієнт» — вибір об’єкта і збільшення interestCount у відповідному Property.

Статистика та запити

- **Середня ціна:** Agency.averagePrice() обчислює середню вартість доступних об’єктів.
- **Популярна локація:** Agency.mostPopularArea() визначає локацію з максимальною кількістю об’єктів.
- **Найбільше кімнат:** Agency.propertyWithMostRooms() повертає Property з максимумом кімнат.
- **Адреси продажу:** Agency.saleAddresses() формує список адрес для статусу НА_ПРОДАЖ.
- **Найбільший попит:** Agency.mostDemandedProperties() сортує об’єкти за interestCount у спадному порядку.

Узгодженість та розширюваність

- **Типобезпека:** використання enum-ів у формах через JComboBox забезпечує валідні значення статусу та типу послуги.
- **Супровід та зрозумілість:** чіткі гетери/сетери, компактні діалоги, повідомлення користувачу та структурований менеджер даних підвищують підтримуваність коду.

Проєкт досягнув усіх поставлених вимог: реалізовано інтуїтивний GUI з трьома вкладками, повний набір CRUD-операцій, експорт у CSV, корисні статистичні запити та базові механізми взаємодії клієнтів із об’єктами.

8.2. Труднощі та їх вирішення

У процесі розробки програми «Агентство нерухомості» виникали певні труднощі, пов'язані як із технічною реалізацією, так і з організацією коду. Нижче наведено основні проблеми та способи їх вирішення.

1. Організація структури класів

- **Труднощі:** на початковому етапі було складно визначити оптимальну структуру класів та їх взаємозв'язки. Спочатку використовувалися лише базові сутності, але згодом виникла потреба у введенні додаткових полів (наприклад, «попит» у Property) та перерахувань (ServiceType, AvailabilityStatus).
- **Вирішення:** було розроблено чітку архітектуру з окремими класами для кожної сутності (Property, Client, Realtor, Agency, RealEstateApp), а також введено enum-и для забезпечення типобезпеки та узгодженості даних.

2. Реалізація графічного інтерфейсу (Swing)

- **Труднощі:** створення трьох вкладок з окремими таблицями вимагало правильної синхронізації даних між GUI та внутрішніми списками у класі Agency. Також виникали проблеми з оновленням таблиць після редагування чи видалення записів.
- **Вирішення:** для кожної вкладки було створено власну DefaultTableModel, а всі CRUD-операції синхронізуються з відповідними списками у Agency. Після редагування чи видалення дані оновлюються безпосередньо у таблиці.

3. Валідація даних у формах

- **Труднощі:** при введенні числових значень (ціна, площа, поверх, рік, кількість кімнат) користувач міг допускати помилки (наприклад, вводити текст замість числа). Це призводило до винятків та збоїв у роботі програми.
- **Вирішення:** у діалогових вікнах було додано обробку винятків (try–catch) та повідомлення користувачу через JOptionPane про некоректність введених даних.

4. Збереження даних у файл CSV

- **Труднощі:** початково виникали проблеми із коректним форматом збереження (наприклад, відсутність деяких полів або неправильне кодування).
- **Вирішення:** було реалізовано метод savePropertiesToFile() у класі Agency, який зберігає всі поля Property, включно з «попитом». Для користувача додано повідомлення про успішне збереження або помилку.

5. Реалізація статистичних запитів

- **Труднощі:** потрібно було забезпечити коректну роботу методів для обчислення середньої ціни, пошуку найпопулярної локації та визначення об'єкта з найбільшою кількістю кімнат.
- **Вирішення:** у класі Agency реалізовано окремі методи (averagePrice(), mostPopularArea(), propertyWithMostRooms(), saleAddresses(), mostDemandedProperties()), які використовують колекції та алгоритми сортування/підрахунку.

6. Позначення зацікавлення клієнтом

- **Труднощі:** потрібно було реалізувати механізм, який дозволяє клієнту позначати зацікавлення у конкретному об'єкті нерухомості, збільшуючи поле interestCount.
- **Вирішення:** у вкладці «Клієнт» додано кнопку «Позначити зацікавлення нерухомістю», яка відкриває діалог вибору об'єкта. Після вибору викликається метод markInterest() у Property, що збільшує значення «Попит».

Підсумок

Усі труднощі, що виникали під час розробки, були успішно подолані завдяки чіткій структуризації коду, використанню винятків для валідації даних, введенню перерахувань для типобезпеки та реалізації окремих методів для статистики й збереження. Це дозволило створити стабільну, функціональну та розширювану програму.

9. Список використаних джерел

1. **Kathy Sierra, Bert Bates. Head First Java. O'Reilly Media.**
   Класична книга для початківців, яка пояснює основи Java у доступній формі з прикладами. (Дата звернення 22.11.2025) - https://dev.to/tarunfulera/10-best-java-programming-books-for-2025-4bom
2. **Herbert Schildt. Java: The Complete Reference, Eleventh Edition. McGraw-Hill Education.**
   Повний довідник по Java, охоплює синтаксис, бібліотеки та практичні приклади (Дата звернення 23.11.2025) - https://top5best.com/java-programming-reference
3. **Oracle. javax.swing (Java Platform SE 8) Documentation.**
   Офіційна документація по бібліотеці Swing для створення графічних інтерфейсів. (Датан звернення 27.11.2025) - https://docs.oracle.com/javase/8/docs/api/javax/swing/package-summary.html
4. **GeeksforGeeks. Java Collections Tutorial.**
   Пояснення роботи з колекціями (List, Set, Map), приклади використання та алгоритми. (Дата звернення 28.11.2025) - https://www.geeksforgeeks.org/java/java-collection-tutorial/
5. **Oracle. Reading, Writing, and Creating Files (The Java™ Tutorials > Essential I/O).**
   Офіційний туторіал по роботі з файлами у Java (читання, запис, створення). (Дата звернення 28.11.2025) - https://docs.oracle.com/javase/tutorial/essential/io/file.html
6. **M. T. Somashekara, D. S. Guru, K. S. Manjunatha. Object Oriented Programming with Java. PHI Learning Pvt. Ltd.**
   Підручник з ООП у Java з прикладами та поясненнями ключових концепцій. (Дата звернення 29.11.2025) - https://books.google.com/books/about/Object_Oriented_Programming_with_Java.html?id=A8wnDwAAQBAJ

7. **GeeksforGeeks. Java Design Patterns Tutorial.**
   Практичний посібник із застосування шаблонів проєктування у Java (Creational, Structural, Behavioral) (Дата звернення 1.12.2025) - https://www.geeksforgeeks.org/system-design/java-design-patterns/
8. **Власні навички.**

10. Додаток А

Лістинг програми

```
// Імпорти для GUI, таблиць, подій, файлів і колекцій
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.BorderLayout;
import java.awt.GridLayout;
import java.awt.event.*;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.*;

// Перерахування: типи послуг та статуси наявності
enum ServiceType { ПРОДАЖ, ОРЕНДА, ПІДБІР }
enum AvailabilityStatus { НА_ПРОДАЖ, ПРОДАНА, В_ОРЕНДІ }

// Інтерфейси для розширення (вивід та фільтрація)
interface Printable { void print(); }
interface Filterable { boolean matches(FilterCriteria criteria); }

// Критерії фільтрації
class FilterCriteria {
   String type, location;
   double minPrice, maxPrice;
   AvailabilityStatus status;

   public FilterCriteria(String type, String location, double minPrice,
double maxPrice, AvailabilityStatus status) {
       this.type = type;
       this.location = location;
       this.minPrice = minPrice;
       this.maxPrice = maxPrice;
       this.status = status;
   }
}

// Клас Рієлтор: дані про співробітника агентства
```

```
class Realtor implements Printable {
   private String firmName;
   private String fullName;
   private String address;
   private String phone;
   private ServiceType serviceType;

   public Realtor(String firmName, String fullName, String address,
String phone, ServiceType serviceType) {
       this.firmName = firmName;
       this.fullName = fullName;
       this.address = address;
       this.phone = phone;
       this.serviceType = serviceType;
   }
   // Гетери/сетери
   public String getFirmName() { return firmName; }
   public String getFullName() { return fullName; }
   public String getAddress() { return address; }
   public String getPhone() { return phone; }
   public ServiceType getServiceType() { return serviceType; }

   public void setFirmName(String v) { firmName = v; }
   public void setFullName(String v) { fullName = v; }
   public void setAddress(String v) { address = v; }
   public void setPhone(String v) { phone = v; }
   public void setServiceType(ServiceType v) { serviceType = v; }

   @Override
   public void print() {
       System.out.println(firmName + " | " + fullName + " | " + address +
" | " + phone + " | " + serviceType);
   }
}

// Клас Клієнт: дані про клієнта агентства
class Client implements Printable {
   private String fullName;
   private String address;
```

```
    private String phone;
    private ServiceType serviceType;

    public Client(String fullName, String address, String phone,
ServiceType serviceType) {
        this.fullName = fullName;
        this.address = address;
        this.phone = phone;
        this.serviceType = serviceType;
    }

    // Гетери/сетери
    public String getFullName() { return fullName; }
    public String getAddress() { return address; }
    public String getPhone() { return phone; }
    public ServiceType getServiceType() { return serviceType; }

    public void setFullName(String v) { fullName = v; }
    public void setAddress(String v) { address = v; }
    public void setPhone(String v) { phone = v; }
    public void setServiceType(ServiceType v) { serviceType = v; }

    @Override
    public void print() {
        System.out.println(fullName + " | " + address + " | " + phone + "
| " + serviceType);
    }
}

// Клас Нерухомість: дані про об'єкт + поле попиту
class Property implements Printable, Filterable {
    private double area;
    private int floor;
    private int rooms;
    private int yearBuilt;
    private double price;
    private AvailabilityStatus availability;
    private String address;
    private String location; // для запиту "популярна локація"
```

```
    private String type;      // тип нерухомості (квартира/будинок тощо)
    private int interestCount = 0; // "попит": збільшується при
зацікавленні клієнта

    public Property(String type, String location, String address, double
price, AvailabilityStatus availability,
                    double area, int floor, int yearBuilt, int rooms) {
        this.type = type;
        this.location = location;
        this.address = address;
        this.price = price;
        this.availability = availability;
        this.area = area;
        this.floor = floor;
        this.yearBuilt = yearBuilt;
        this.rooms = rooms;
    }

    // Гетери/сетери
    public String getType() { return type; }
    public String getLocation() { return location; }
    public String getAddress() { return address; }
    public double getPrice() { return price; }
    public AvailabilityStatus getAvailability() { return availability; }
    public double getArea() { return area; }
    public int getFloor() { return floor; }
    public int getYearBuilt() { return yearBuilt; }
    public int getRooms() { return rooms; }
    public int getInterestCount() { return interestCount; }

    public void setType(String v) { type = v; }
    public void setLocation(String v) { location = v; }
    public void setAddress(String v) { address = v; }
    public void setPrice(double v) { price = v; }
    public void setAvailability(AvailabilityStatus v) { availability = v;
}
    public void setArea(double v) { area = v; }
    public void setFloor(int v) { floor = v; }
    public void setYearBuilt(int v) { yearBuilt = v; }
```

```
    public void setRooms(int v) { rooms = v; }

    // Метод для позначення попиту
    public void markInterest() { interestCount++; }

    @Override
    public boolean matches(FilterCriteria c) {
        if (c == null) return true;
        boolean okType = c.type == null || c.type.isEmpty() ||
type.equalsIgnoreCase(c.type);
        boolean okLoc  = c.location == null || c.location.isEmpty() ||
location.equalsIgnoreCase(c.location);
        boolean okPrice = (c.minPrice <= 0 || price >= c.minPrice) &&
(c.maxPrice <= 0 || price <= c.maxPrice);
        boolean okStatus = c.status == null || availability == c.status;
        return okType && okLoc && okPrice && okStatus;
    }

    @Override
    public void print() {
        System.out.println(type + ", " + location + ", " + address + ", "
+ price + ", " + availability);
    }
}

// Менеджер даних: списки і запити/статистика
class Agency {
    private String name;

    List<Property> properties = new ArrayList<>();
    List<Client> clients = new ArrayList<>();
    List<Realtor> realtors = new ArrayList<>();

    public Agency(String name) { this.name = name; }
    public String getName() { return name; }

    // CRUD для нерухомості
    public void addProperty(Property p) { properties.add(p); }
```

```
    public void removeProperty(int index) { if (index >= 0 && index <
properties.size()) properties.remove(index); }

    // CRUD для клієнтів
    public void addClient(Client c) { clients.add(c); }
    public void removeClient(int index) { if (index >= 0 && index <
clients.size()) clients.remove(index); }

    // CRUD для рієлторів
    public void addRealtor(Realtor r) { realtors.add(r); }
    public void removeRealtor(int index) { if (index >= 0 && index <
realtors.size()) realtors.remove(index); }

    // Збереження нерухомості у CSV
    public void savePropertiesToFile(String filename) {
        try (PrintWriter writer = new PrintWriter(new
FileWriter(filename))) {
            for (Property p : properties) {
                writer.println(p.getType() + "," + p.getLocation() + "," +
p.getAddress() + "," + p.getPrice() + "," +
                        p.getAvailability() + "," + p.getArea() + "," +
p.getFloor() + "," + p.getYearBuilt() + "," +
                        p.getRooms() + "," + p.getInterestCount());
            }
            JOptionPane.showMessageDialog(null, "Дані нерухомості
збережено у файл: " + filename);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(null, "Помилка запису у файл.");
        }
    }

    // Запити/статистика
    public double averagePrice() {
        if (properties.isEmpty()) return 0;
        double sum = 0;
        for (Property p : properties) sum += p.getPrice();
        return sum / properties.size();
    }
```

```
    public String mostPopularArea() {
        Map<String, Integer> count = new HashMap<>();
        for (Property p : properties) {
            count.put(p.getLocation(), count.getOrDefault(p.getLocation(),
0) + 1);
        }
        return count.isEmpty() ? "Немає даних"
                : Collections.max(count.entrySet(),
Map.Entry.comparingByValue()).getKey();
    }

    public Property propertyWithMostRooms() {
        if (properties.isEmpty()) return null;
        return Collections.max(properties,
Comparator.comparingInt(Property::getRooms));
    }

    public List<String> saleAddresses() {
        List<String> result = new ArrayList<>();
        for (Property p : properties) {
            if (p.getAvailability() == AvailabilityStatus.НА_ПРОДАЖ) {
                result.add(p.getAddress());
            }
        }
        return result;
    }

    public List<Property> mostDemandedProperties() {
        List<Property> copy = new ArrayList<>(properties);

copy.sort(Comparator.comparingInt(Property::getInterestCount).reversed())
;
        return copy;
    }
}
// Головний клас з графічним інтерфейсом (три вкладки: Рієлтор, Клієнт,
Нерухомість)
public class RealEstateApp {
    private final Agency agency = new Agency("Нерухомість Плюс");
```

```
    private JFrame frame;

    // Таблиці для кожної вкладки
    private DefaultTableModel realtorModel;
    private JTable realtorTable;

    private DefaultTableModel clientModel;
    private JTable clientTable;

    private DefaultTableModel propertyModel;
    private JTable propertyTable;

    public RealEstateApp() {
        frame = new JFrame("Агентство нерухомості — " + agency.getName());
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(1000, 600);

        // Вкладки для трьох сутностей
        JTabbedPane tabs = new JTabbedPane();
        tabs.addTab("Рієлтор", buildRealtorPanel());
        tabs.addTab("Клієнт", buildClientPanel());
        tabs.addTab("Нерухомість", buildPropertyPanel());

        // Меню запитів
        JMenuBar menuBar = new JMenuBar();
        JMenu reports = new JMenu("Запити");
        JMenuItem miRealtors = new JMenuItem("Список рієлторів");
        miRealtors.addActionListener(e -> showRealtorsList());
        JMenuItem miClients = new JMenuItem("Список клієнтів");
        miClients.addActionListener(e -> showClientsList());
        JMenuItem miSaleAddresses = new JMenuItem("Список адрес продажу");
        miSaleAddresses.addActionListener(e -> showSaleAddresses());
        JMenuItem miMostRooms = new JMenuItem("Нерухомість з найбільшою
кількістю кімнат");
        miMostRooms.addActionListener(e -> showPropertyWithMostRooms());
        JMenuItem miAvgPrice = new JMenuItem("Середня вартість
нерухомості");
        miAvgPrice.addActionListener(e -> showAveragePrice());
```

```
        JMenuItem miDemand = new JMenuItem("Об'єкти з найбільшим
попитом");
        miDemand.addActionListener(e -> showMostDemandedProperties());

        reports.add(miRealtors);
        reports.add(miClients);
        reports.add(miSaleAddresses);
        reports.add(miMostRooms);
        reports.add(miAvgPrice);
        reports.add(miDemand);
        menuBar.add(reports);

        frame.setJMenuBar(menuBar);
        frame.add(tabs, BorderLayout.CENTER);
        frame.setVisible(true);
    }

    // Панель "Рієлтор": таблиця + кнопки CRUD
    private JPanel buildRealtorPanel() {
        realtorModel = new DefaultTableModel(new Object[]{
                "Назва фірми", "ПІБ", "Адреса", "Телефон", "Тип послуги"
        }, 0);
        realtorTable = new JTable(realtorModel);
        JScrollPane scroll = new JScrollPane(realtorTable);

        // Кнопки для роботи з рієлторами
        JButton add = new JButton("Додати рієлтора");
        add.addActionListener(e -> openRealtorDialog(null, -1));

        JButton edit = new JButton("Редагувати");
        edit.addActionListener(e -> {
            int row = realtorTable.getSelectedRow();
            if (row >= 0) {
                Realtor r = agency.realtors.get(row);
                openRealtorDialog(r, row);
            } else JOptionPane.showMessageDialog(frame, "Оберіть
рієлтора.");
        });
```

```
        JButton delete = new JButton("Видалити");
        delete.addActionListener(e -> {
            int row = realtorTable.getSelectedRow();
            if (row >= 0) {
                agency.removeRealtor(row);
                realtorModel.removeRow(row);
            } else JOptionPane.showMessageDialog(frame, "Оберіть
рієлтора.");
        });

        JPanel controls = new JPanel();
        controls.add(add); controls.add(edit); controls.add(delete);

        JPanel panel = new JPanel(new BorderLayout());
        panel.add(scroll, BorderLayout.CENTER);
        panel.add(controls, BorderLayout.SOUTH);
        return panel;
    }

    // Діалог додавання/редагування рієлтора
    private void openRealtorDialog(Realtor existing, int row) {
        JDialog d = new JDialog(frame, existing == null ? "Новий рієлтор"
: "Редагувати рієлтора", true);
        d.setSize(400, 300);
        d.setLayout(new GridLayout(6, 2));

        JTextField firm = new JTextField(existing == null ? "" :
existing.getFirmName());
        JTextField name = new JTextField(existing == null ? "" :
existing.getFullName());
        JTextField address = new JTextField(existing == null ? "" :
existing.getAddress());
        JTextField phone = new JTextField(existing == null ? "" :
existing.getPhone());

        JComboBox<ServiceType> service = new
JComboBox<>(ServiceType.values());
        if (existing != null)
service.setSelectedItem(existing.getServiceType());
```

```
        d.add(new JLabel("Назва фірми:")); d.add(firm);
        d.add(new JLabel("ПІБ:")); d.add(name);
        d.add(new JLabel("Адреса:")); d.add(address);
        d.add(new JLabel("Телефон:")); d.add(phone);
        d.add(new JLabel("Тип послуги:")); d.add(service);

        JButton ok = new JButton("Зберегти");
        ok.addActionListener(e -> {
            try {
                if (existing == null) {
                    Realtor r = new Realtor(firm.getText(),
name.getText(), address.getText(), phone.getText(),
                            (ServiceType) service.getSelectedItem());
                    agency.addRealtor(r);
                    realtorModel.addRow(new Object[]{
                            r.getFirmName(), r.getFullName(),
r.getAddress(), r.getPhone(), r.getServiceType()
                    });
                } else {
                    existing.setFirmName(firm.getText());
                    existing.setFullName(name.getText());
                    existing.setAddress(address.getText());
                    existing.setPhone(phone.getText());
                    existing.setServiceType((ServiceType)
service.getSelectedItem());

                    realtorModel.setValueAt(existing.getFirmName(), row,
0);
                    realtorModel.setValueAt(existing.getFullName(), row,
1);
                    realtorModel.setValueAt(existing.getAddress(), row,
2);
                    realtorModel.setValueAt(existing.getPhone(), row, 3);
                    realtorModel.setValueAt(existing.getServiceType(),
row, 4);
                }
                d.dispose();
            } catch (Exception ex) {
```

```
                JOptionPane.showMessageDialog(d, "Перевірте коректність
даних.");
            }
        });
        d.add(new JLabel()); d.add(ok);
        d.setVisible(true);
    }

    // Панель "Клієнт": таблиця + кнопки CRUD + позначення попиту
    private JPanel buildClientPanel() {
        clientModel = new DefaultTableModel(new Object[]{
                "ПІБ", "Адреса", "Телефон", "Тип послуги"
        }, 0);
        clientTable = new JTable(clientModel);
        JScrollPane scroll = new JScrollPane(clientTable);

        // Кнопки для роботи з клієнтами
        JButton add = new JButton("Додати клієнта");
        add.addActionListener(e -> openClientDialog(null, -1));

        JButton edit = new JButton("Редагувати");
        edit.addActionListener(e -> {
            int row = clientTable.getSelectedRow();
            if (row >= 0) {
                Client c = agency.clients.get(row);
                openClientDialog(c, row);
            } else JOptionPane.showMessageDialog(frame, "Оберіть
клієнта.");
        });

        JButton delete = new JButton("Видалити");
        delete.addActionListener(e -> {
            int row = clientTable.getSelectedRow();
            if (row >= 0) {
                agency.removeClient(row);
                clientModel.removeRow(row);
            } else JOptionPane.showMessageDialog(frame, "Оберіть
клієнта.");
        });
```

```
        // Додаткова кнопка: позначення зацікавлення нерухомістю
        JButton interest = new JButton("Позначити зацікавлення
нерухомістю");
        interest.addActionListener(e -> markInterestFlow());

        JPanel controls = new JPanel();
        controls.add(add); controls.add(edit); controls.add(delete);
controls.add(interest);

        JPanel panel = new JPanel(new BorderLayout());
        panel.add(scroll, BorderLayout.CENTER);
        panel.add(controls, BorderLayout.SOUTH);
        return panel;
    }

    // Діалог додавання/редагування клієнта
    private void openClientDialog(Client existing, int row) {
        JDialog d = new JDialog(frame, existing == null ? "Новий клієнт" :
"Редагувати клієнта", true);
        d.setSize(400, 300);
        d.setLayout(new GridLayout(5, 2));

        JTextField name = new JTextField(existing == null ? "" :
existing.getFullName());
        JTextField address = new JTextField(existing == null ? "" :
existing.getAddress());
        JTextField phone = new JTextField(existing == null ? "" :
existing.getPhone());
        JComboBox<ServiceType> service = new
JComboBox<>(ServiceType.values());
        if (existing != null)
service.setSelectedItem(existing.getServiceType());

        d.add(new JLabel("ПІБ:")); d.add(name);
        d.add(new JLabel("Адреса:")); d.add(address);
        d.add(new JLabel("Телефон:")); d.add(phone);
        d.add(new JLabel("Тип послуги:")); d.add(service);
```

```
        JButton ok = new JButton("Зберегти");
        ok.addActionListener(e -> {
            try {
                if (existing == null) {
                    Client c = new Client(name.getText(),
address.getText(), phone.getText(),
                            (ServiceType) service.getSelectedItem());
                    agency.addClient(c);
                    clientModel.addRow(new Object[]{ c.getFullName(),
c.getAddress(), c.getPhone(), c.getServiceType() });
                } else {
                    existing.setFullName(name.getText());
                    existing.setAddress(address.getText());
                    existing.setPhone(phone.getText());
                    existing.setServiceType((ServiceType)
service.getSelectedItem());

                    clientModel.setValueAt(existing.getFullName(), row,
0);
                    clientModel.setValueAt(existing.getAddress(), row, 1);
                    clientModel.setValueAt(existing.getPhone(), row, 2);
                    clientModel.setValueAt(existing.getServiceType(), row,
3);
                }
                d.dispose();
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(d, "Перевірте коректність
даних.");
            }
        });
        d.add(new JLabel()); d.add(ok);
        d.setVisible(true);
    }

    // Метод позначення попиту: клієнт обирає об'єкт нерухомості
    private void markInterestFlow() {
        if (agency.properties.isEmpty()) {
            JOptionPane.showMessageDialog(frame, "Немає нерухомості для
позначення зацікавлення.");
```

```
            return;
        }
        // Формуємо список доступних об'єктів
        String[] items = new String[agency.properties.size()];
        for (int i = 0; i < items.length; i++) {
            Property p = agency.properties.get(i);
            items[i] = p.getType() + " | " + p.getAddress() + " | " +
p.getPrice() + "₴";
        }
        // Діалог вибору об'єкта
        String choice = (String) JOptionPane.showInputDialog(frame,
"Оберіть об'єкт:",
                "Зацікавлення", JOptionPane.PLAIN_MESSAGE, null, items,
items[0]);
        if (choice != null) {
            int idx = Arrays.asList(items).indexOf(choice);
            agency.properties.get(idx).markInterest();
            JOptionPane.showMessageDialog(frame, "Зацікавлення
зафіксовано.");
        }
    }

    // Панель "Нерухомість": таблиця + кнопки CRUD + збереження у файл
    private JPanel buildPropertyPanel() {
        propertyModel = new DefaultTableModel(new Object[]{
                "Тип", "Локація", "Адреса", "Ціна", "Статус", "Площа",
"Поверх", "Рік", "Кімнат", "Попит"
        }, 0);
        propertyTable = new JTable(propertyModel);
        JScrollPane scroll = new JScrollPane(propertyTable);

        // Кнопки для роботи з об'єктами нерухомості
        JButton add = new JButton("Додати об'єкт");
        add.addActionListener(e -> openPropertyDialog(null, -1));

        JButton edit = new JButton("Редагувати");
        edit.addActionListener(e -> {
            int row = propertyTable.getSelectedRow();
            if (row >= 0) {
```

```
                Property p = agency.properties.get(row);
                openPropertyDialog(p, row);
            } else JOptionPane.showMessageDialog(frame, "Оберіть
об'єкт.");
        });

        JButton delete = new JButton("Видалити");
        delete.addActionListener(e -> {
            int row = propertyTable.getSelectedRow();
            if (row >= 0) {
                agency.removeProperty(row);
                propertyModel.removeRow(row);
            } else JOptionPane.showMessageDialog(frame, "Оберіть
об'єкт.");
        });

        JButton save = new JButton("Зберегти у файл (CSV)");
        save.addActionListener(e ->
agency.savePropertiesToFile("properties_data.csv"));

        JPanel controls = new JPanel();
        controls.add(add); controls.add(edit); controls.add(delete);
controls.add(save);

        JPanel panel = new JPanel(new BorderLayout());
        panel.add(scroll, BorderLayout.CENTER);
        panel.add(controls, BorderLayout.SOUTH);
        return panel;
    }

    // Діалог додавання/редагування об'єкта нерухомості
    private void openPropertyDialog(Property existing, int row) {
        JDialog d = new JDialog(frame, existing == null ? "Новий об'єкт" :
"Редагувати об'єкт", true);
        d.setSize(450, 450);
        d.setLayout(new GridLayout(11, 2));

        JTextField type = new JTextField(existing == null ? "" :
existing.getType());
```

```
        JTextField location = new JTextField(existing == null ? "" :
existing.getLocation());
        JTextField address = new JTextField(existing == null ? "" :
existing.getAddress());
        JTextField price = new JTextField(existing == null ? "" :
String.valueOf(existing.getPrice()));
        JComboBox<AvailabilityStatus> status = new
JComboBox<>(AvailabilityStatus.values());
        if (existing != null)
status.setSelectedItem(existing.getAvailability());

        JTextField area = new JTextField(existing == null ? "" :
String.valueOf(existing.getArea()));
        JTextField floor = new JTextField(existing == null ? "" :
String.valueOf(existing.getFloor()));
        JTextField year = new JTextField(existing == null ? "" :
String.valueOf(existing.getYearBuilt()));
        JTextField rooms = new JTextField(existing == null ? "" :
String.valueOf(existing.getRooms()));

        d.add(new JLabel("Тип:")); d.add(type);
        d.add(new JLabel("Локація:")); d.add(location);
        d.add(new JLabel("Адреса:")); d.add(address);
        d.add(new JLabel("Ціна:")); d.add(price);
        d.add(new JLabel("Статус:")); d.add(status);
        d.add(new JLabel("Площа:")); d.add(area);
        d.add(new JLabel("Поверх:")); d.add(floor);
        d.add(new JLabel("Рік побудови:")); d.add(year);
        d.add(new JLabel("Кімнат:")); d.add(rooms);

        JButton ok = new JButton("Зберегти");
        ok.addActionListener(e -> {
            try {
                String t = type.getText();
                String loc = location.getText();
                String addr = address.getText();
                double pr = Double.parseDouble(price.getText());
                AvailabilityStatus st = (AvailabilityStatus)
status.getSelectedItem();
```

```
                double ar = Double.parseDouble(area.getText());
                int fl = Integer.parseInt(floor.getText());
                int yr = Integer.parseInt(year.getText());
                int rm = Integer.parseInt(rooms.getText());

                if (existing == null) {
                    Property p = new Property(t, loc, addr, pr, st, ar,
fl, yr, rm);
                    agency.addProperty(p);
                    propertyModel.addRow(new Object[]{
                            p.getType(), p.getLocation(), p.getAddress(),
p.getPrice(), p.getAvailability(),
                            p.getArea(), p.getFloor(), p.getYearBuilt(),
p.getRooms(), p.getInterestCount()
                    });
                } else {
                    existing.setType(t);
                    existing.setLocation(loc);
                    existing.setAddress(addr);
                    existing.setPrice(pr);
                    existing.setAvailability(st);
                    existing.setArea(ar);
                    existing.setFloor(fl);
                    existing.setYearBuilt(yr);
                    existing.setRooms(rm);

                    propertyModel.setValueAt(existing.getType(), row, 0);
                    propertyModel.setValueAt(existing.getLocation(), row,
1);
                    propertyModel.setValueAt(existing.getAddress(), row,
2);
                    propertyModel.setValueAt(existing.getPrice(), row, 3);
                    propertyModel.setValueAt(existing.getAvailability(),
row, 4);
                    propertyModel.setValueAt(existing.getArea(), row, 5);
                    propertyModel.setValueAt(existing.getFloor(), row, 6);
                    propertyModel.setValueAt(existing.getYearBuilt(), row,
7);
                    propertyModel.setValueAt(existing.getRooms(), row, 8);
```

```
                    propertyModel.setValueAt(existing.getInterestCount(),
row, 9);
                }
                d.dispose();
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(d, "Перевірте коректність
числових полів.");
            }
        });
        d.add(new JLabel()); d.add(ok);
        d.setVisible(true);
    }

    // Вивід списку рієлторів
    private void showRealtorsList() {
        if (agency.realtors.isEmpty()) {
JOptionPane.showMessageDialog(frame, "Список рієлторів порожній.");
return; }
        StringBuilder sb = new StringBuilder("Рієлтори:\n");
        for (Realtor r : agency.realtors) sb.append("-
").append(r.getFirmName()).append(" | ").append(r.getFullName())
                .append(" | ").append(r.getPhone()).append(" |
").append(r.getServiceType()).append("\n");
        JOptionPane.showMessageDialog(frame, sb.toString());
    }

    // Вивід списку клієнтів
    private void showClientsList() {
        if (agency.clients.isEmpty()) {
JOptionPane.showMessageDialog(frame, "Список клієнтів порожній.");
return; }
        StringBuilder sb = new StringBuilder("Клієнти:\n");
        for (Client c : agency.clients) sb.append("-
").append(c.getFullName()).append(" | ").append(c.getPhone())
                .append(" | ").append(c.getServiceType()).append("\n");
        JOptionPane.showMessageDialog(frame, sb.toString());
    }

    // Вивід списку адрес продажу (тільки доступні об'єкти)
```

```
    private void showSaleAddresses() {
        List<String> addresses = agency.saleAddresses();
        if (addresses.isEmpty()) { JOptionPane.showMessageDialog(frame,
"Немає адрес для продажу."); return; }
        StringBuilder sb = new StringBuilder("Адреси продажу:\n");
        for (String a : addresses) sb.append("- ").append(a).append("\n");
        JOptionPane.showMessageDialog(frame, sb.toString());
    }

    // Вивід нерухомості з найбільшою кількістю кімнат
    private void showPropertyWithMostRooms() {
        Property p = agency.propertyWithMostRooms();
        if (p == null) { JOptionPane.showMessageDialog(frame, "Немає
даних."); return; }
        JOptionPane.showMessageDialog(frame, "Найбільше кімнат:\n" +
                p.getRooms() + " — " + p.getType() + " | " +
p.getAddress());
    }

    // Вивід середньої вартості нерухомості
    private void showAveragePrice() {
        double avg = agency.averagePrice();
        JOptionPane.showMessageDialog(frame, "Середня вартість
нерухомості: " + String.format("%.2f", avg) + "₴");
    }

    // Вивід об'єктів з найбільшим попитом (сортування за interestCount)
    private void showMostDemandedProperties() {
        List<Property> list = agency.mostDemandedProperties();
        if (list.isEmpty()) { JOptionPane.showMessageDialog(frame, "Немає
даних."); return; }
        StringBuilder sb = new StringBuilder("Найбільший попит
(зацікавлення):\n");
        for (Property p : list) {
            sb.append("- ").append(p.getInterestCount()).append(" |
").append(p.getType())
                    .append(" | ").append(p.getAddress()).append("\n");
        }
        JOptionPane.showMessageDialog(frame, sb.toString());
```

```
    }

    // Запуск програми
    public static void main(String[] args) {
        SwingUtilities.invokeLater(RealEstateApp::new);
    }
}
```